DBMS

TEXT BOOKS

- 1. Database Management Systems, **Raghurama Krishnan**, Johannes Gehrke, TATA McGrawHill 3rd edition
- 2. Database System Concepts, **Silberschatz**, Korth, McGraw hill, 5 th edition.

- **REFERENCES :**
- 1. Database Systems design, Implementation, and Management, Peter Rob & Carlos Coronel 7 th Edition.
- 2. Fundamentals of Database Systems, Elmasri Navate, Pearson Education
- 3. Introduction to Database Systems, C.J.Date, Pearson Education

HISTORY of DBMS

• 1950s and early 1960s: Magnetic tapes were developed for data storage. Data processing tasks such as payroll were automated, with data stored on tapes. Processing of data consisted of reading data from one or more tapes and writing data to a new tape. Data could also be input from punched card decks, and output to printers.

For example, salary raises were processed by entering the raises on punched cards and reading the punched card deck in synchronization with a tape containing the master salary details. The records had to be in the same sorted order. The salary raises would be added to the salary read from the master tape, and written to a new tape; the new tape would become the new master tape.

Tapes (and card decks) could be read only sequentially, and data sizes were much larger than main memory; thus, data processing programs were forced to process data in a particular order, by reading and merging data from tapes and card decks.

• Late 1960s and 1970s: Widespread use of hard disks in the late 1960s changed the scenario for data processing greatly, since hard disks allowed direct access to data. The position of data on disk was immaterial, since any location on disk could be accessed in just tens of milliseconds. Data were thus freed from the tyranny of sequentiality.

With disks, network and hierarchical databases could be created that allowed data structures such as lists and trees to be stored on disk. Programmers could construct and manipulate these data structures.

A landmark paper by Codd [1970] defined the relational model and nonprocedural ways of querying data in the relational model, and relational databases were born. The simplicity of the relational model and the possibility of hiding implementation details completely from the programmer were enticing indeed.

Codd later won the prestigious Association of Computing Machinery Turing Award for his work.

• 1980s: Although academically interesting, the relational model was not used in practice initially, because of its perceived performance disadvantages; relational databases could not match the performance of existing network and hierarchical databases. That changed with System R, a groundbreaking project at IBM Research that developed techniques for the construction of an efficient relational database system.

The fully functional System R prototype led to IBM's first relational database product. At the same time, the Ingres system was being developed at the University of California at Berkeley. It led to a commercial product of the same name. Initial commercial relational database systems, such as IBM DB2 , Oracle, Ingres, and DEC Rdb, played a major role in advancing techniques for efficient processing of declarative queries. By the early 1980s, relational databases had become competitive with network and hierarchical database systems even in the area of performance. Relational databases were so easy to use that they eventually replaced network and hierarchical databases;

Most importantly, they had to keep efficiency in mind when designing their programs, which involved a lot of effort. In contrast, in a relational database, almost all these low-level tasks are carried out automatically by the database, leaving the programmer free to work at a logical level. Since attaining dominance in the 1980s, the relational model has reigned supreme among data models. The 1980s also saw much research on parallel and distributed databases, as well as initial work on object-oriented databases.

• Early 1990s: The SQL language was designed primarily for decision support applications, which are query-intensive, yet the mainstay of databases in the 1980s was transaction-processing applications, which are update-intensive.
Decision support and querying re-emerged as a major application area for databases. Tools for analyzing large amounts of data saw large growths in usage. Many database vendors introduced parallel database products in this period. Database vendors also began to add object-relational support to their databases.

• 1990s: The major event of the 1990s was the explosive growth of the World Wide Web. Databases were deployed much more extensively than ever before. Database systems now had to support very high transaction-processing rates, as well as very high reliability and 24 × 7 availability (availability 24 hours a day, 7 days a week, meaning no downtime for scheduled maintenance activities). Database systems also had to support Web interfaces to data.

• 2000s: The first half of the 2000s saw the emerging of XML and the associated query language XQuery as a new database technology. Although XML is widely used for data exchange, as well as for storing certain complex data types, relational databases still form the core of a vast majority of large-scale database applications. In this time period we have also witnessed the growth in “autonomic-computing/auto-admin” techniques for minimizing system administration effort. This period also saw a significant growth in use of open-source database systems, particularly Postgre SQL and My SQL.

The latter part of the decade has seen growth in specialized databases for data analysis, in particular column-stores, which in effect store each column of a table as a separate array, and highly parallel database systems designed for analysis of very large data sets. Several novel distributed data-storage systems have been built to handle the data management requirements of very large Web sites such as Amazon, Facebook, Google, Microsoft and Yahoo!, and some of these are now offered as Web services that can be used by application developers.

Database and DBMS

- **Database**:

  A very large collection (of files) of related data

- Examples: Accounts in a bank, student database, Airline reservations… also, facebook pictures and comments, web logs, etc…
- Models a real world enterprise:
  - Entities (e.g., teams, games / students, courses)
  - Relationships (e.g., student takes CS460)

- **Data Base Management System** (DBMS):

  A software package/system that can be used to store, manage and retrieve data from databases that persist for long periods of time!

- Examples: Oracle, IBM DB2, MS SQLServer, MySQL, PostgreSQL, SQLite,…

- Database System: DBMS+data (+ applications)

DBMS Vs Files

- In the early days, database applications were built on top of file systems
- Drawbacks of using file systems to store data:
  - Data redundancy and inconsistency
    - Multiple file formats, duplication of information in different files
  - Difficulty in accessing data
    - Need to write a new program to carry out each new task
  - Integrity problems
    - Integrity constraints (e.g. account balance > 0) become part of program code
    - Hard to add new constraints or change existing ones

- Atomicity of updates
  - Failures may leave database in an inconsistent state with partial updates carried out
  - E.g. transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
  - Concurrent accesses needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - E.g. two people reading a balance and updating it at the same time
- Security problems

- DBMS offer automated solutions to all the above problems; they solve problems caused by different people writing different applications independently.

Advantages of Using DBMS

1- Controlling Redundancy:

DBMS has the ability to control redundancy which improves the performance of the query.

Advantages of Using DBMS

2- Restricting Unauthorized Access:

Users or user groups are given account numbers protected by passwords, which they can use to gain access to the database through security and authorization subsystem

Advantages of Using DBMS

3- Providing Persistent Storage for Program Objects and Data Structures:

Programming languages typically have complex data structures, such as record types in PASCAL or class definitions in C++. The values of program variables are discarded once a program terminates, unless the programmer explicitly stores them in permanent files, which often involves converting these complex structures into a format suitable for file storage

Advantages of Using DBMS

4 - Providing Multiple User Interfaces:

A DBMS provides a variety of user interfaces:

- query languages for casual users;
- programming language interfaces for application programmers;
- forms and command codes for parametric users;
- and menu-driven interfaces and natural language interfaces for stand-alone users.

Advantages of Using DBMS

5 - Enforcing Integrity Constraints:

The simplest type involves specifying a data type for each data item, and a more complex type involves specifying that a record in one file must be related to records in other files.

Advantages of Using DBMS

6 - Providing Backup and Recovery:

Provide facilities for recovering from hardware or software failures.

Data Independence

- One big problem in application development is the *separation* of applications from data
- Do I have to change my program when I …
  - replace my hard drive?
  - store the data in a b-tree instead of a hash file?
  - partition the data into two physical files (or merge two physical files into one)?
  - store salary as floating point number instead of integer?
  - develop other applications that use the same set of data?
  - add more data fields to support other applications?

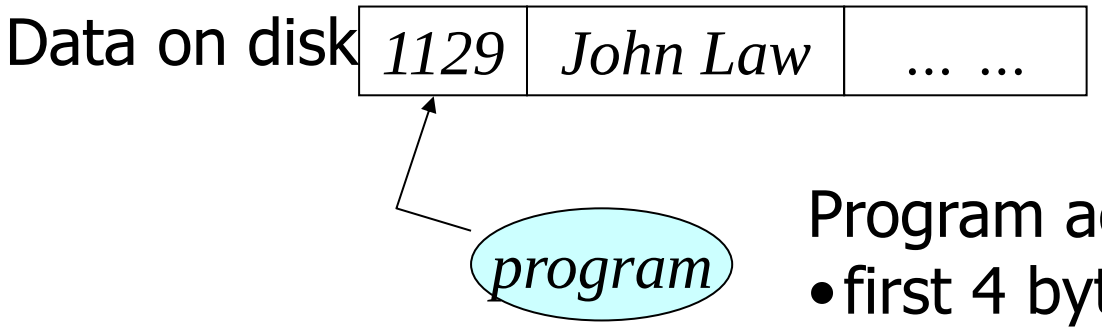
- Ability to modify a schema definition in one level without affecting a schema definition in the next higher level.

- The interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.

- Two levels of data independence:

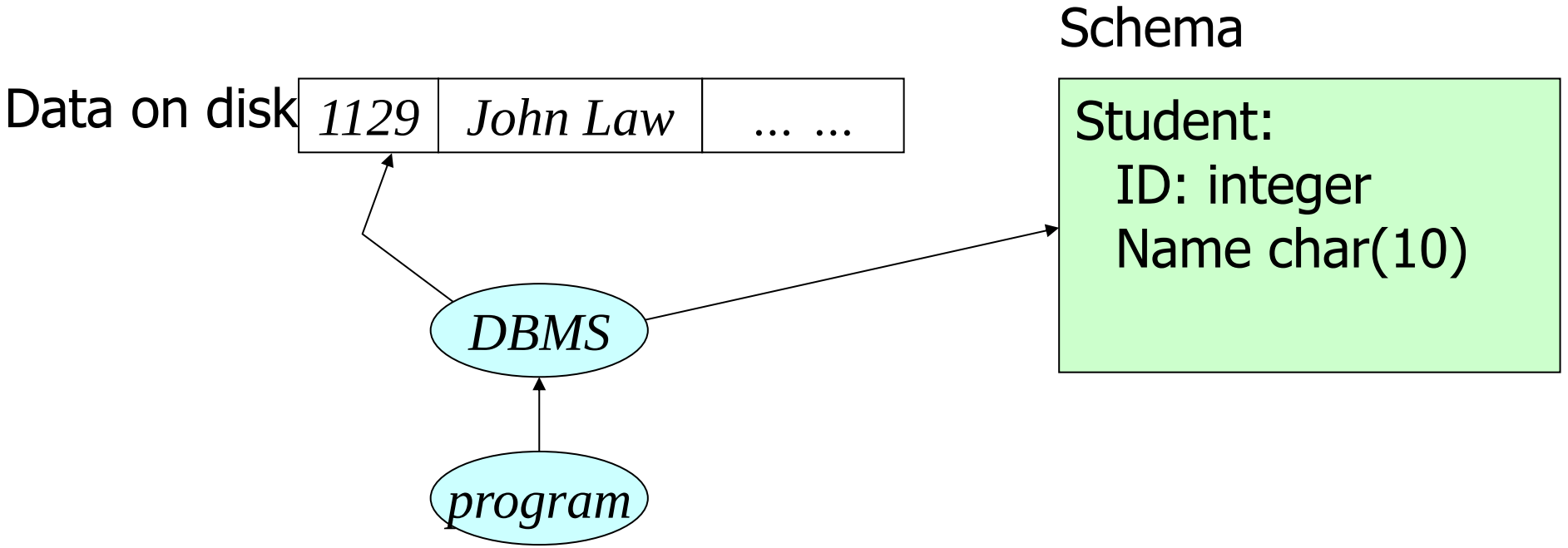
  - Physical data independence (users are shielded from changes in the physical structure of the data)

  - Logical data independence (users are shielded from changes in the logical structure of the data)



Program accessing data directly has to know:

- first 4 bytes is an ID number
- next 10 bytes is an employee name



Data Models

The data models can be classified into four different categories:

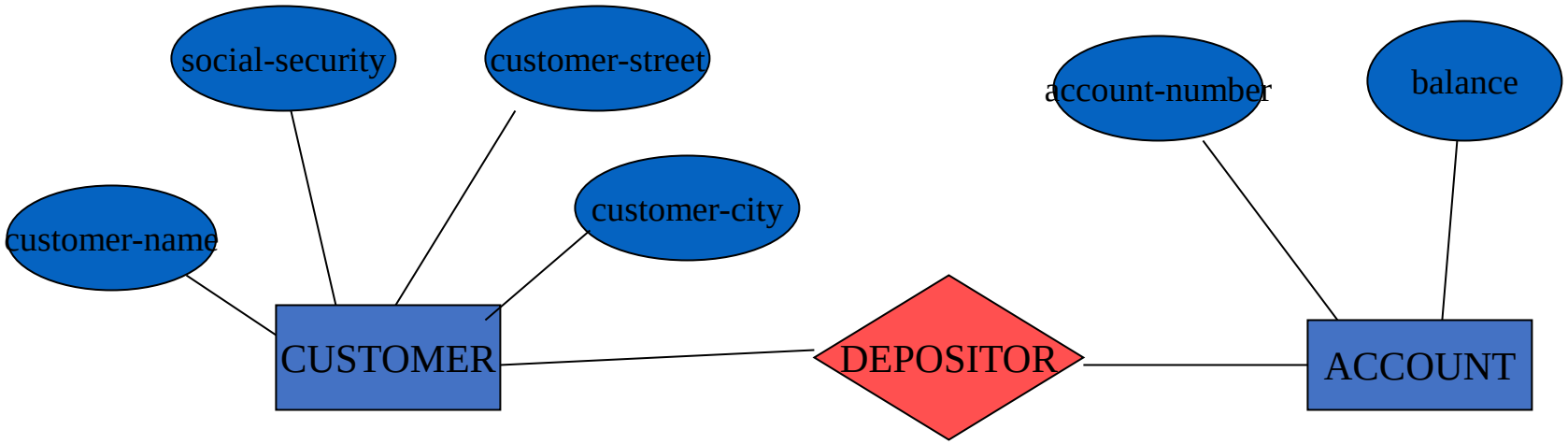
- **Relational Model**. The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Tables are also known

as relations. The relational model is an example of a record-based model. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type. The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model.

customer-name	social-security	customer-street	customer-city	account-number
Johnson	192-83-7465	Alma	Palo Alto	A-101
Smith	019-28-3746	North	Rye	A-215
Johnson	192-83-7465	Alma	Palo Alto	A-201
Jones	321-12-3123	Main	Harrison	A-217
Smith	019-28-3746	North	Rye	A-201

account-number	balance
A-101	500
A-201	900
A-215	700
A-217	750

• **Entity-Relationship Model**. The entity-relationship (E-R) data model uses a collection of basic objects, called entities, and relationships among these objects. An entity is a “thing” or “object” in the real world that is distinguishable from other objects. The entity-relationship model is widely used in database design.



• **Object-Based Data Model**. Object-oriented programming (especially in Java, C++, or C#) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity. The object-relational data model combines features of the object-oriented data model and relational data model.

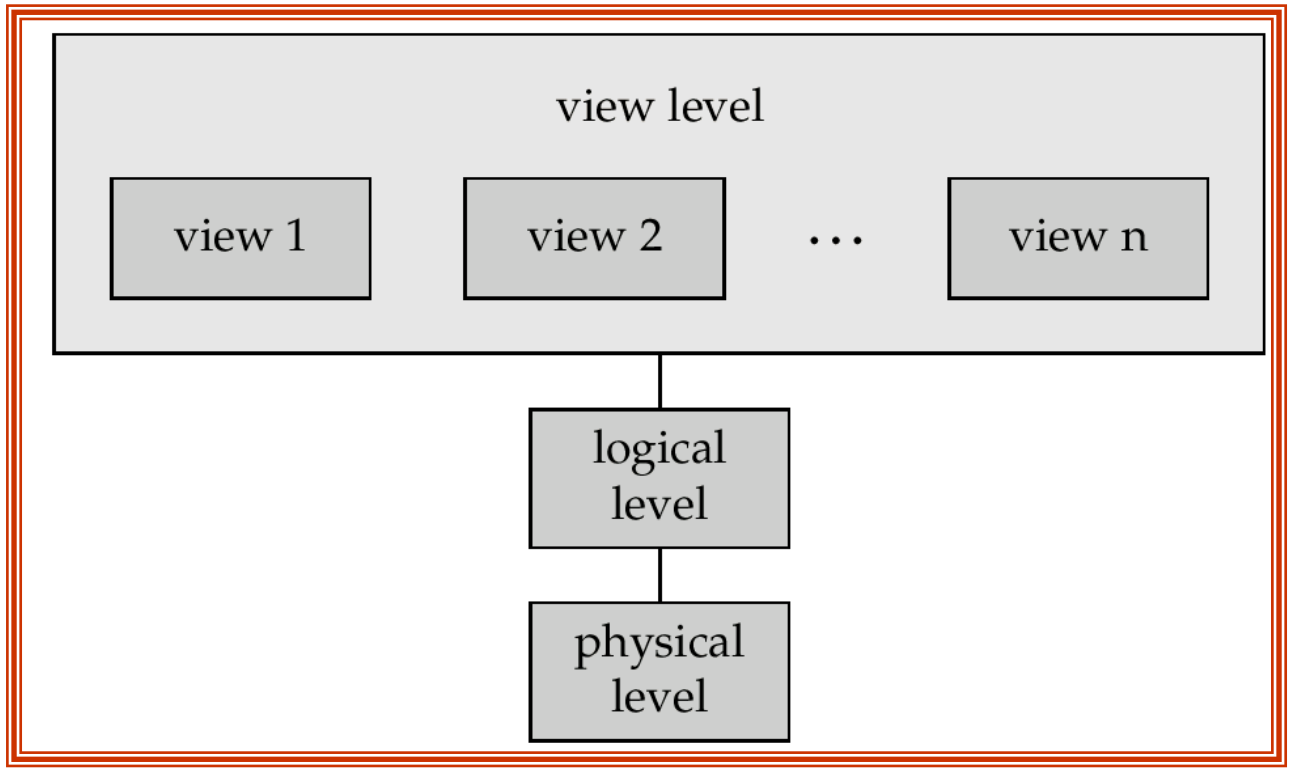
• **Semistructured Data Model**. The semistructured data model permits the specification of data where individual data items of the same type may have different sets of attributes. This is in contrast to the data models mentioned earlier, where every data item of a particular type must have the same set of attributes. The Extensible Markup Language (XML) is widely used to represent semistructured data.

Levels of Abstraction

For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

- **Physical level.** The lowest level of abstraction describes how the data are actually stored. The physical level describes complex low-level data structures in detail.
- **Logical level.** The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.
- **View level**. The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database.

An architecture for a database system



Imagine a university wants to create a database to manage student information. In the conceptual design phase, the primary concern is identifying the main entities and their relationships within the university context. Key entities might include students, courses, instructors, and departments. Relationships could include a student enrolling in courses, instructors teaching courses, and departments managing courses.

Example:

- Entities: Student, Course, Instructor, Department
- Relationships: Student enrolls in Course, Instructor teaches Course, Department manages Course

Continuing with our university example, in the logical design phase, we would define attributes for each entity and specify their data types, primary keys, and foreign keys. This stage also involves normalizing the data to eliminate redundancy and ensure data integrity.

Example:

- Student Entity:
  - Attributes: StudentID (Primary Key), FirstName, LastName, DateOfBirth
- Course Entity:
  - Attributes: CourseID (Primary Key), CourseName, Credits
- Instructor Entity:
  - Attributes: InstructorID (Primary Key), FirstName, LastName
- Department Entity:
  - Attributes: DepartmentID (Primary Key), DepartmentName

For our university database, in the physical design phase, we would determine which DBMS to use (e.g., MySQL, Oracle, PostgreSQL) and create the actual database schema. This involves specifying the exact table structures, data types, constraints, and indexes. It also includes decisions about data storage, partitioning, and access control.

Example:

•Student Table (MySQL Syntax):

sql

•CREATE TABLE Student (
StudentID INT PRIMARY KEY,
FirstName VARCHAR(50),
LastName VARCHAR(50),
DateOfBirth DATE
);

•Course Table:

sql

CREATE TABLE Course (
CourseID INT PRIMARY KEY,
CourseName VARCHAR(100),
Credits INT
);

Structure of DBMS

The structure of DBMS is divided into three components namely "Query Processor", "Storage Manager", and "Disk Storage".

Query Processor

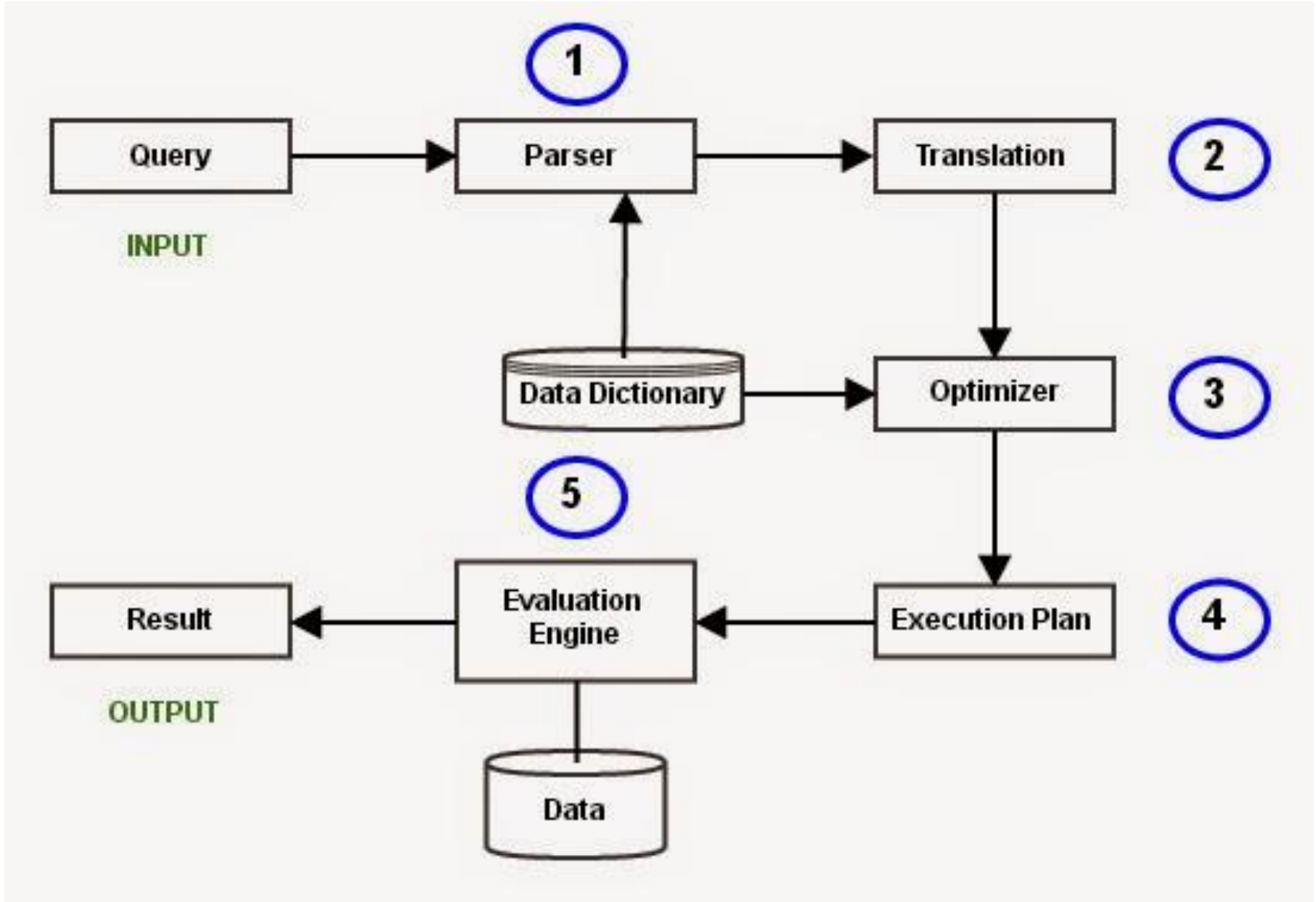
- DML Compiler
- DDL Interpreter
- Embedded DML Pre-compiler
- Query Evaluation Engine

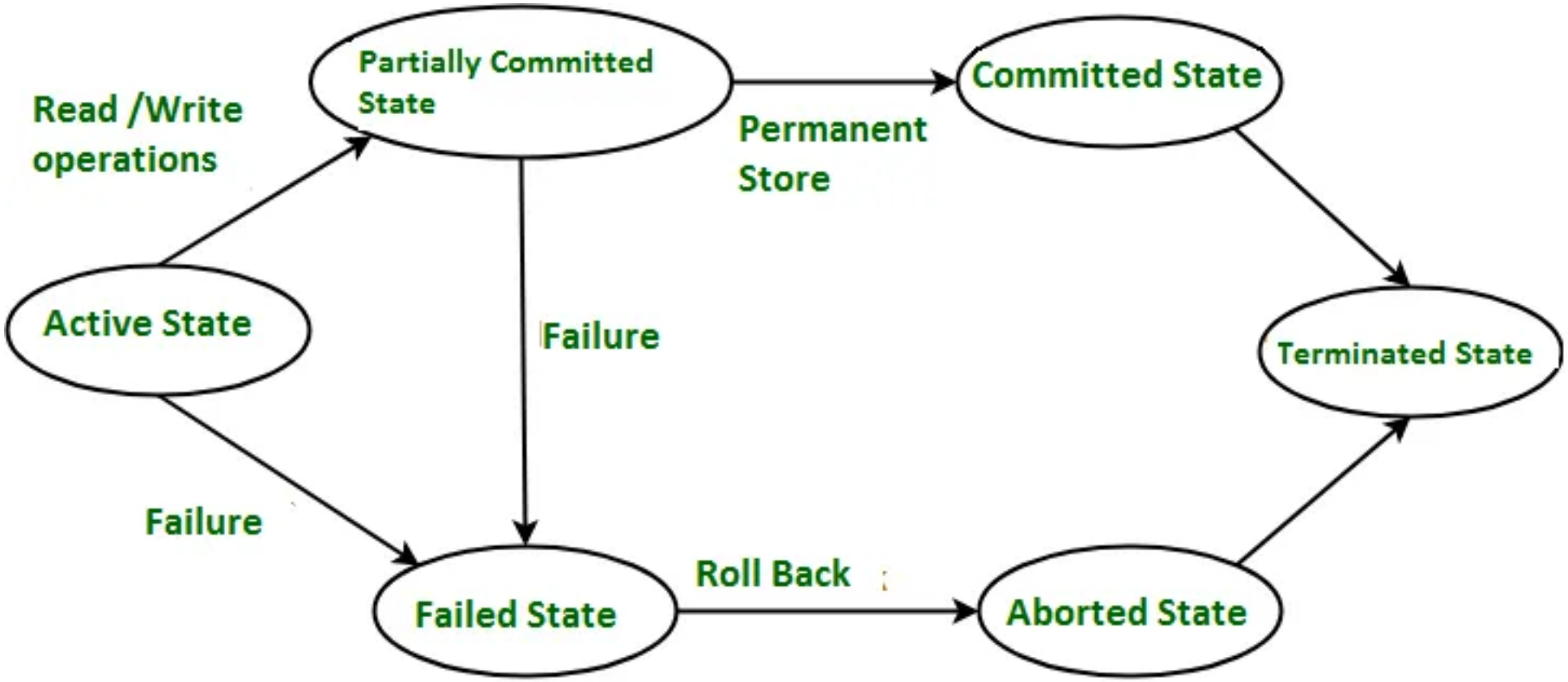
Storage Manager

- Authorization and Integrity Manager
- Transaction Manager
- File Manager
- Buffer Manager

Disk Storage

- Data Files
- Data Dictionary
- Indices
- Statistical Data


1
Query
Parser
Translation
2
INPUT
Data Dictionary
Optimizer
3
5
Result
Evaluation Engine
Execution Plan
4
OUTPUT
Data



Transaction States in DBMS

A simplified architecture for a database system

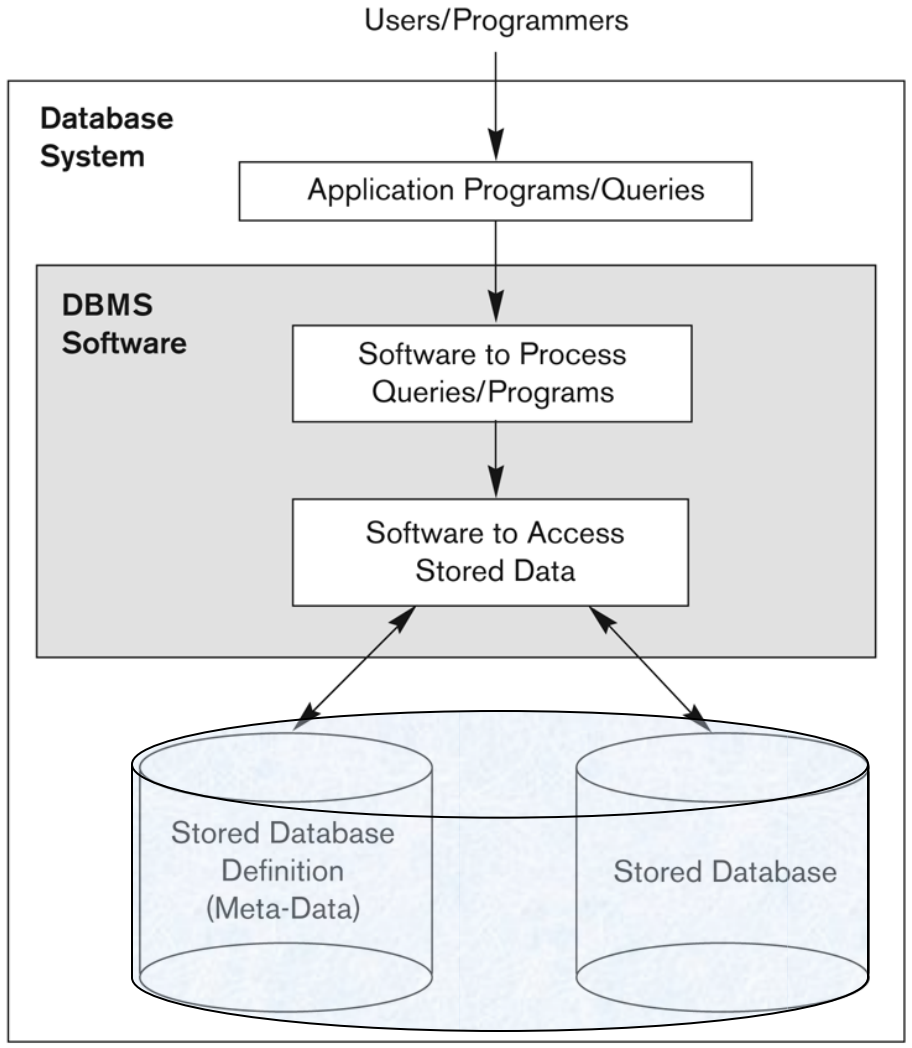
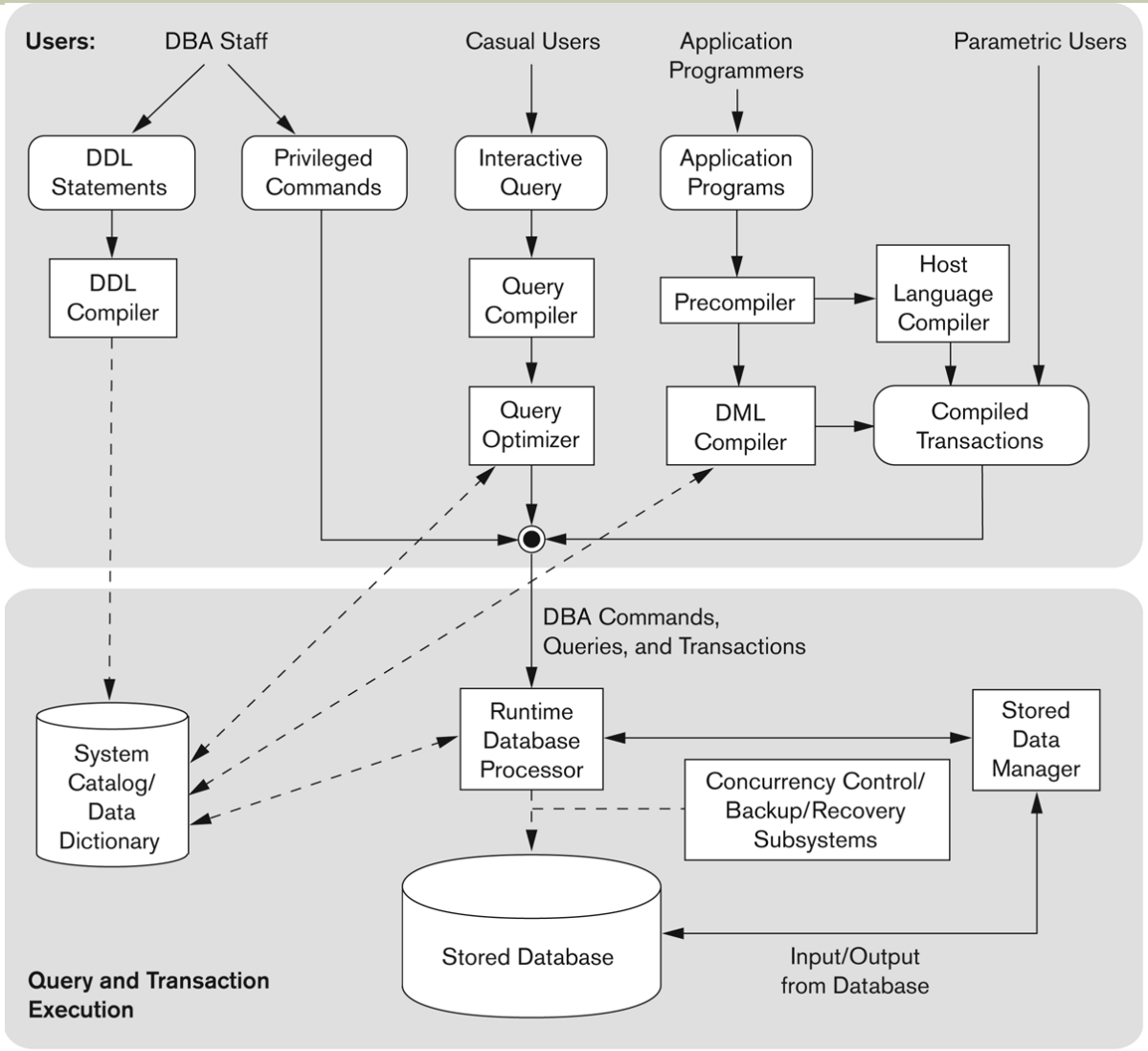


Figure 1.1
A simplified database system environment.

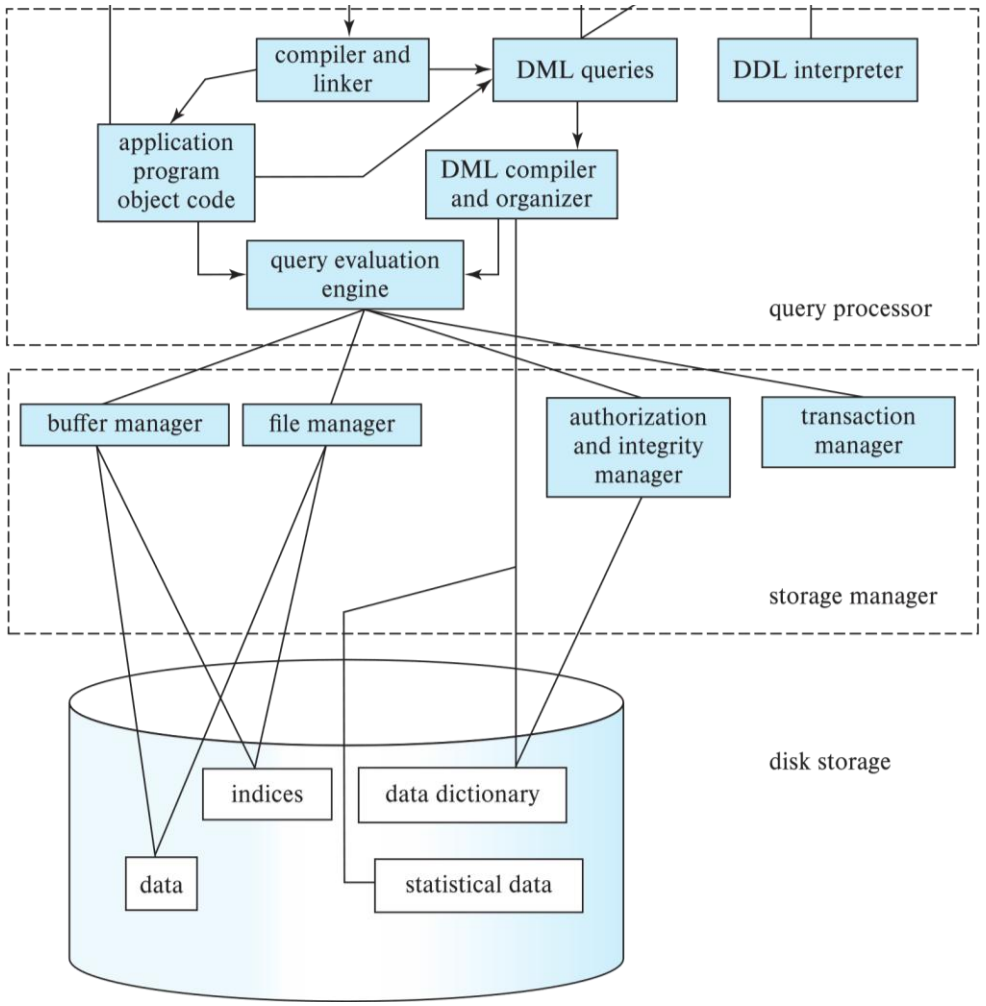
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

A simplified architecture for a database system



Copyright © 2016 Ramez Elmasri a

A simplified architecture for a database system



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database Users

- Users may be divided into
  - Those who actually use and control the database content, and those who design, develop and maintain database applications (called "*Actors on the Scene*"), and
  - Those who design and develop the DBMS software and related tools, and the computer systems operators (called "*Workers Behind the Scene*").

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database Users – Actors on the Scene

- Actors on the scene
  - Database administrators
    - Responsible for authorizing access to the database, for coordinating and monitoring its use, acquiring software and hardware resources, controlling its use and monitoring efficiency of operations.
  - Database designers
    - Responsible to define the content, the structure, the constraints, and functions or transactions against the database. They must communicate with the end-users and understand their needs.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database End Users

- Actors on the scene (continued)
  - **End-users:** They use the data for queries, reports and some of them update the database content. End-users can be categorized into:
    - **Casual**: access database occasionally when needed
    - **Naïve** or parametric: they make up a large section of the end-user population.
      - They use previously well-defined functions in the form of "canned transactions" against the database.
      - Users of mobile apps mostly fall in this category
      - Bank-tellers or reservation clerks are parametric users who do this activity for an entire shift of operations.
      - Social media users post and read information from websites

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database End Users (continued)

- Sophisticated:
  - These include business analysts, scientists, engineers, others thoroughly familiar with the system capabilities.
  - Many use tools in the form of software packages that work closely with the stored database.
- Stand-alone:
  - Mostly maintain personal databases using ready-to-use packaged applications.
  - An example is the user of a tax program that creates its own internal database.
  - Another example is a user that maintains a database of personal photos and videos.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database Users – Actors on the Scene (continued)

- System analysts and application developers
  - System analysts: They understand the user requirements of naïve and sophisticated users and design applications including canned transactions to meet those requirements.
  - Application programmers: Implement the specifications developed by analysts and test and debug them before deployment.
  - Business analysts: There is an increasing need for such people who can analyze vast amounts of business data and real-time data ("Big Data") for better decision making related to planning, advertising, marketing etc.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database Users – Actors behind the Scene

- **System designers and implementors:** Design and implement DBMS packages in the form of modules and interfaces and test and debug them. The DBMS must interface with applications, language compilers, operating system components, etc.
- **Tool developers**: Design and implement software systems called tools for modeling and designing databases, performance monitoring, prototyping, test data generation, user interface creation, simulation etc. that facilitate building of applications and allow using database effectively.
- **Operators and maintenance personnel:** They manage the actual running and maintenance of the database system hardware and software environment.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Data Definition Language (DDL)

DDL stands for **D**ata **D**efinition **L**anguage. It is used to define database structure or pattern.

- It is used to create schema, tables, indexes, constraints, etc. in the database.
- Using the DDL statements, you can create the skeleton of the database.
- Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.

Some tasks that come under DDL:

- **Create:** It is used to create objects in the database.
- **Alter:** It is used to alter the structure of the database.
- **Drop:** It is used to delete objects from the database.
- **Truncate:** It is used to remove all records from a table.
- **Rename:** It is used to rename an object.
- **Comment:** It is used to comment on the data dictionary.

These commands are used to update the database schema that's why they come under Data definition language.

Data Manipulation Language (DML)

DML stands for **D**ata **M**anipulation **L**anguage. It is used for accessing and manipulating data in a database. It handles user requests.

- **Select:** It is used to retrieve data from a database.
- **Insert:** It is used to insert data into a table.
- **Update:** It is used to update existing data within a table.
- **Delete:** It is used to delete all records from a table.
- **Merge:** It performs UPSERT operation, i.e., insert or update operations.
- **Call:** It is used to call a structured query language or a Java subprogram.
- **Explain Plan:** It has the parameter of explaining data.
- **Lock Table:** It controls concurrency.

Data Control Language (DCL)

•**DCL** stands for **D**ata **C**ontrol **L**anguage. It is used to retrieve the stored or saved data.

•The DCL execution is transactional. It also has rollback parameters.

(But in Oracle database, the execution of data control language does not have the feature of rolling back.)

Some tasks that come under DCL:

•**Grant:** It is used to give user access privileges to a database.

•**Revoke:** It is used to take back permissions from the user.

There are the following operations which have the authorization of Revoke:

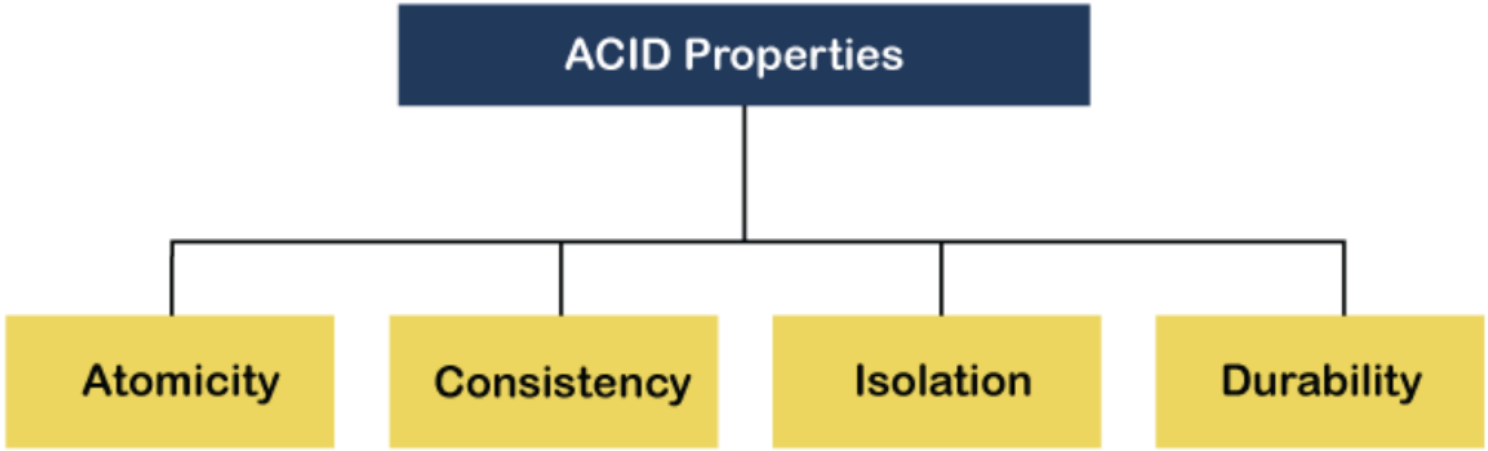
CONNECT, INSERT, USAGE, EXECUTE, DELETE, UPDATE and SELECT.

Transaction Control Language (TCL)

TCL is used to run the changes made by the DML statement. TCL can be grouped into a logical transaction.

Some tasks that come under TCL:

- **Commit:** It is used to save the transaction on the database.
- **Rollback:** It is used to restore the database to original since the last Commit.



Atomicity

The term atomicity defines that the data remains atomic. It means if any operation is performed on the data, either it should be performed or executed completely or should not be executed at all. It further means that the operation should not break in between or execute partially. In the case of executing operations on the transaction, the operation should be completely executed and not partially.

Consistency

The word **consistency** means that the value should remain preserved always. In DBMS, the integrity of the data should be maintained, which means if a change in the database is made, it should remain preserved always. In the case of transactions, the integrity of the data is very essential so that the database remains consistent before and after the transaction. The data should always be correct.

Isolation

The term 'isolation' means separation. In DBMS, Isolation is the property of a database where no data should affect the other one and may occur concurrently. In short, the operation on one database should begin when the operation on the first database gets complete. It means if two operations are being performed on two different databases, they may not affect the value of one another. In the case of transactions, when two or more transactions occur simultaneously, the consistency should remain maintained. Any changes that occur in any particular transaction will not be seen by other transactions until the change is not committed in the memory.

Durability

Durability ensures the permanency of something. In DBMS, the term durability ensures that the data after the successful execution of the operation becomes permanent in the database. The durability of the data should be so perfect that even if the system fails or leads to a crash, the database still survives. However, if it gets lost, it becomes the responsibility of the recovery manager for ensuring the durability of the database. For committing the values, the COMMIT command must be used every time we make changes.

What is an ER Model?

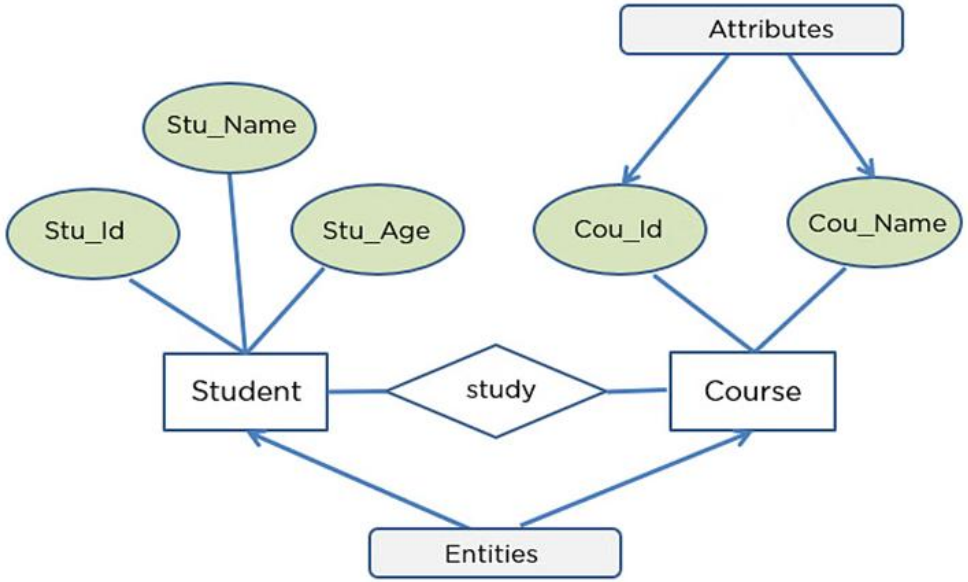
An Entity-Relationship Model represents the structure of the database with the help of a diagram. ER Modelling is a systematic process to design a database, it helps to analyze all data requirements before implementing the database.

History of ER models

Peter Chen proposed ER Diagrams in 1971 to create a uniform convention that can be used as a conceptual modeling tool. Many models were presented and discussed, but none were suitable. The data structure diagrams offered by Charles Bachman also inspired his model.

What is an ER Diagram?

An Entity Relationship Diagram (ER Diagram) pictorially explains the relationship between entities to be stored in a database. Fundamentally, the ER Diagram is a structural design of the database. It acts as a framework created with specialized symbols for the purpose of defining the relationship between the database entities. ER diagram is created based on three principal components: entities, attributes, and relationships.

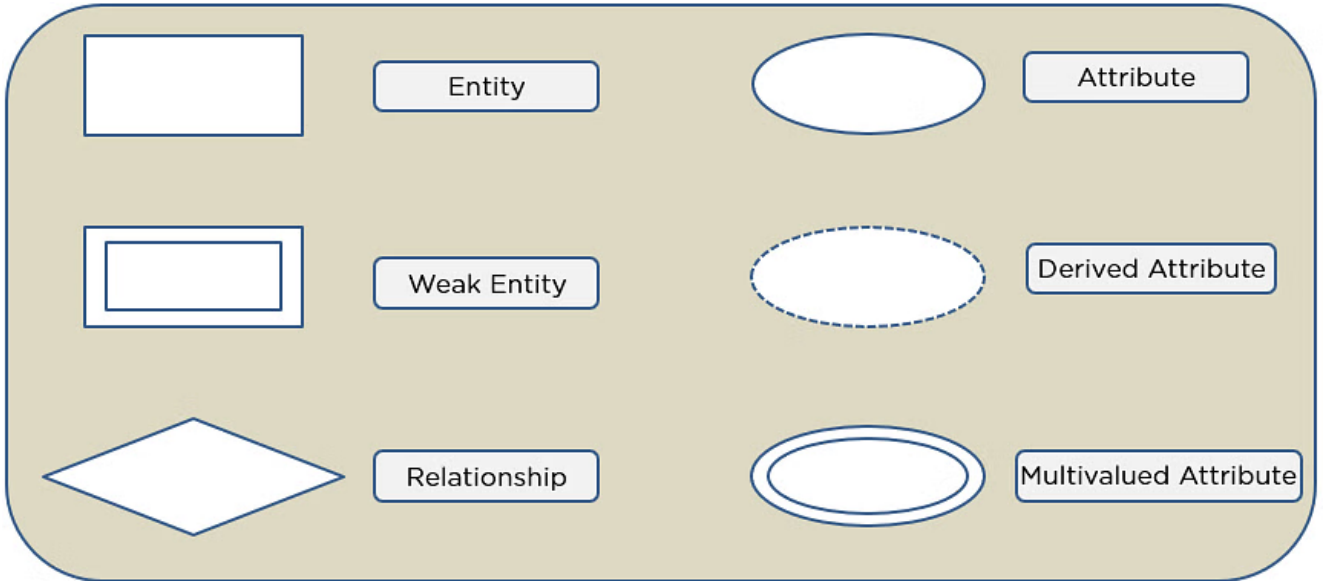


Why Use ER Diagrams in DBMS?

- ER Diagram helps conceptualize the database and helps in finding which fields need to be embedded for a particular entity
- ER Diagram gives a better understanding of the information to be stored in a database
- It reduces complexity and allows database designers to build databases quickly
- It helps to describe elements using Entity-Relationship models
- It allows users to get a preview of the logical structure of the database

Symbols Used in ER Diagrams

- Rectangles: This Entity Relationship Diagram symbol represents entity types
- Ellipses: This symbol represents attributes
- Diamonds: This symbol represents relationship types
- Lines: It links attributes to entity types and entity types with other relationship types
- Primary key: Here, it underlines the attributes
- Double Ellipses: Represents multi-valued attributes



Components of ER Diagram

Three basic concepts:

- **Entities**
  - Weak Entity
- **Attributes**
  - Key Attribute
  - Composite Attribute
  - Multivalued Attribute
  - Derived Attribute
- **Relationships**
  - One-to-One Relationships
  - One-to-Many Relationships
  - Many-to-One Relationships
  - Many-to-Many Relationships

How to Draw an ER Diagram?

Some important points to draw ER diagram:

•First, identify all the Entities. Embed all the entities in a rectangle and label them properly.

•Identify relationships between entities and connect them using a diamond in the middle, illustrating the relationship. Do not connect relationships with each other.

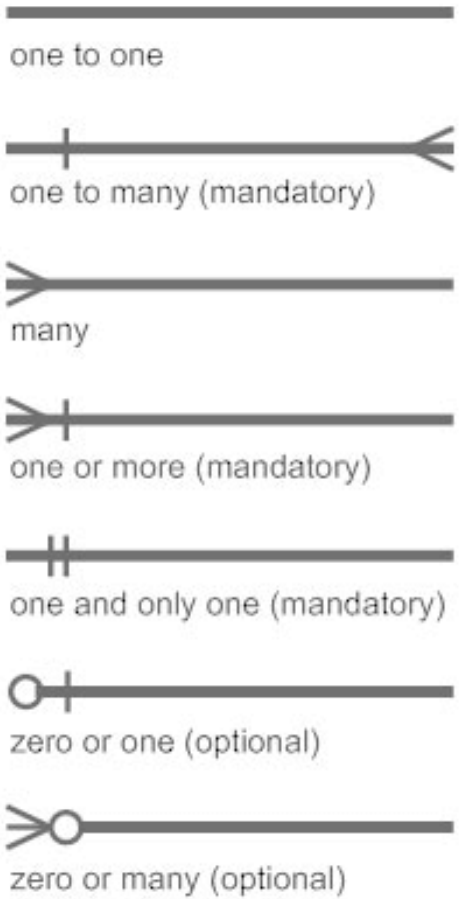
•Connect attributes for entities and label them properly.

•Eradicate any redundant entities or relationships.

•Make sure the ER Diagram supports all the data provided to design the database.

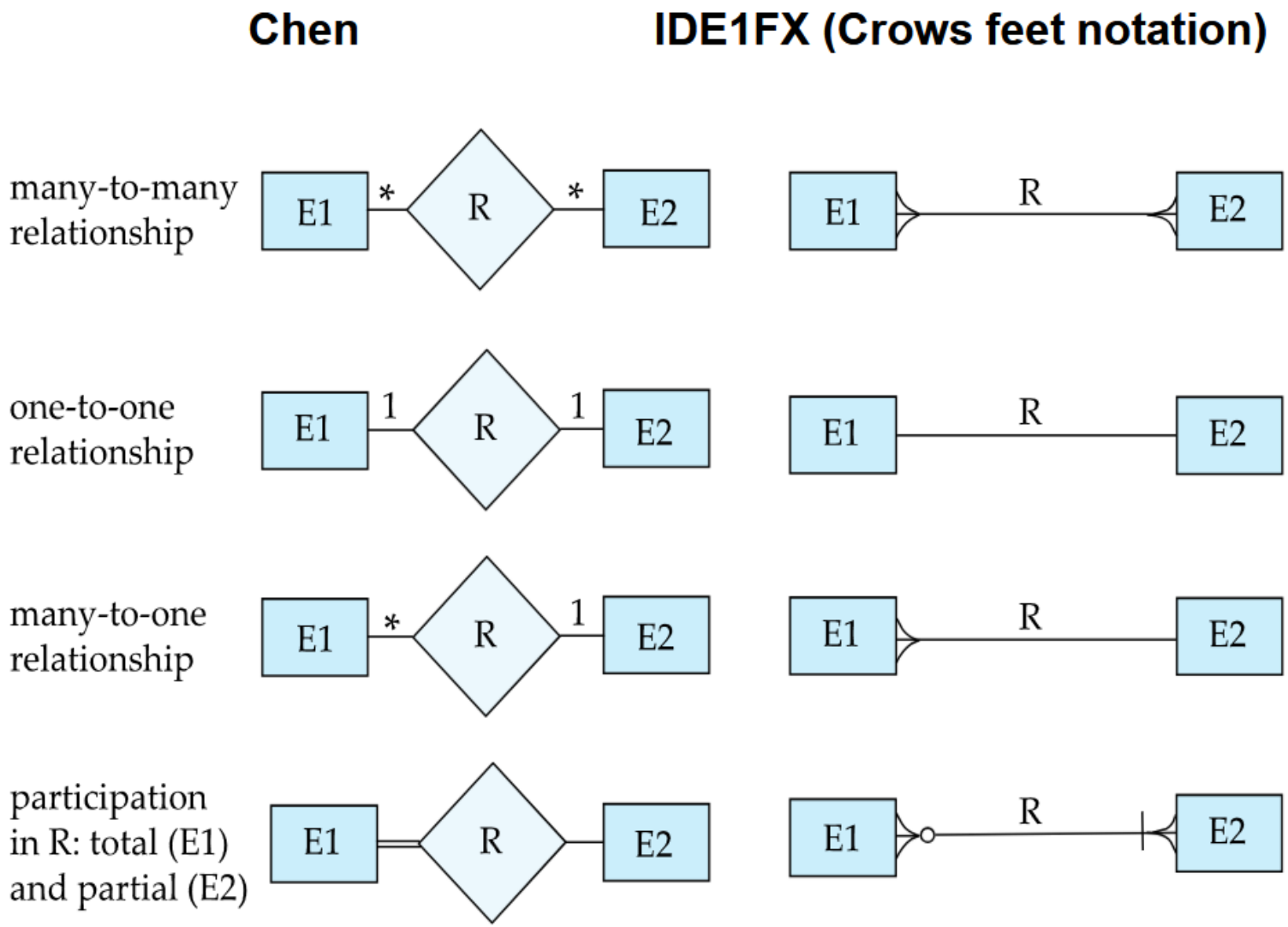
•Effectively use colors to highlight key areas in the diagrams.

Notation of ER diagram





Alternative ER Notations



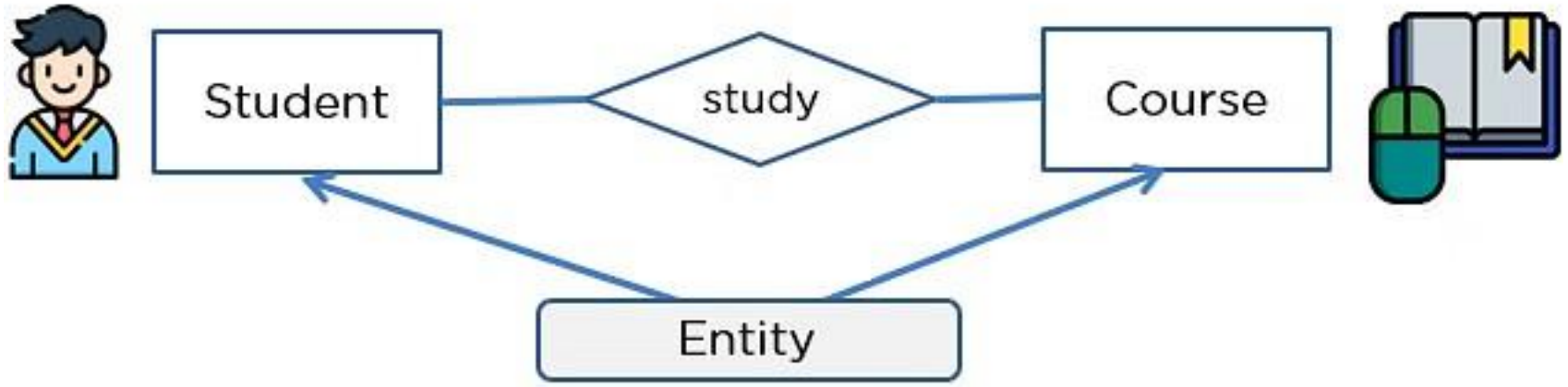
Entities

An *entity* is an object in the real world with an independent existence that can be differentiated from other objects. An entity might be

- An object with physical existence (e.g., a lecturer, a student, a car)
- An object with conceptual existence (e.g., a course, a job, a position)

An entity is represented as a rectangle in an ER diagram.

For example, in a student study course, both the student and the course are entities.



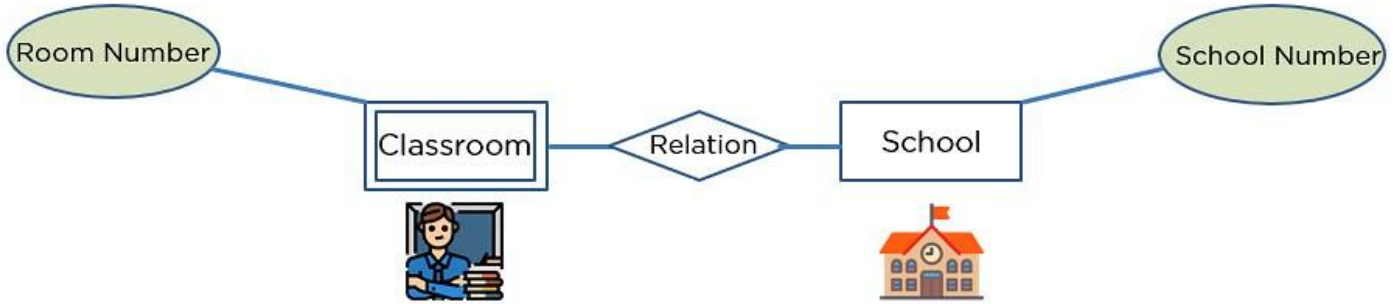
Weak Entity

Entities can be classified based on their strength. An entity is considered weak if its tables are existence dependent.

- That is, it cannot exist without a relationship with another entity
- Its primary key is derived from the primary key of the parent entity

The weak entity is shown as a double rectangle in ER Diagram.

In the example below, school is a strong entity because it has a primary key attribute - school number. Unlike school, the classroom is a weak entity because it does not have any primary key and the room number here acts only as a discriminator.



Independent entities (strong)

Independent entities, also referred to as kernels, are the backbone of the database. They are what other tables are based on. *Kernels* have the following characteristics:

- They are the building blocks of a database.
- The primary key may be simple or composite.
- The primary key is not a foreign key.
- They do not depend on another entity for their existence.

Examples:

- **Student** (Student_ID, Name, Email, Major)
- **Faculty** (Faculty_ID, Name, Department, Email)
- **Course** (Course_ID, Course_Name, Credits, Department)

Dependent entities (weak)

Dependent entities, also referred to as *derived entities*, depend on other tables for their meaning. These entities have the following characteristics:

- Dependent entities are used to connect two kernels together.
- They are said to be existence dependent on two or more tables.
- Many to many relationships become associative tables with at least two foreign keys.
- They may contain other attributes.
- The foreign key identifies each associated table.

- There are three options for the primary key:
  - Use a composite of foreign keys of associated tables if unique
  - Use a composite of foreign keys and a qualifying column
  - Create a new simple primary key

Examples:

- **Enrollment** (Enrollment_ID, Student_ID, Course_ID, Semester)
  - A student **enrolls in** a course, but an enrollment record **cannot exist without** a student and a course.
- **Course Sections** (Section_ID, Course_ID, Faculty_ID, Room_Number, Schedule)
  - A **section** exists only if there is a **course** and a **faculty member** assigned to teach it.

Characteristic entities (associative)

Characteristic entities provide more information about another table. These entities have the following characteristics:

- They represent multivalued attributes.
- They describe other entities.
- They typically have a one to many relationship.
- The foreign key is used to further identify the characterized table.

- Options for primary key are as follows:
  - Use a composite of foreign key plus a qualifying column
  - Create a new simple primary key.

Examples:

- **Grades** (Student_ID, Course_ID, Semester, Grade)
  - This entity **associates** Students and Courses while storing **extra information (grades)**.
- **Faculty Assignments** (Faculty_ID, Course_ID, Semester, Role)
  - This entity captures **which faculty member is teaching which course** in a given semester.

If we consider a COMPANY database, examples of an _____ entity include the Customer table, Employee table or Product table.

The Spouse table, in the COMPANY database, is a _____ entity.

What is Entity Set?

- An entity set is a comprehensive representation of all entities of the same type at a specific time.
- The use of an entity set helps to aggregate and thus manage similar entities within databases.
- The entities of the entity-set have common attributes for each entity, but the values of those attributes are different.

For instance, a record of students in a “Student” table is by each student and the total record or collection would form an entity set. However, entity sets are not static because they have the potential to grow and shrink with the addition or removal of entities.

Example :

•The collection of all the students from the student table at a particular instant of time is an example of an entity set.

•The collection of all the employees from the employee table at a particular instant of time is an example of an entity set.

Entity sets need not be disjoint.

Entity Sets -- *instructor* and *student*

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

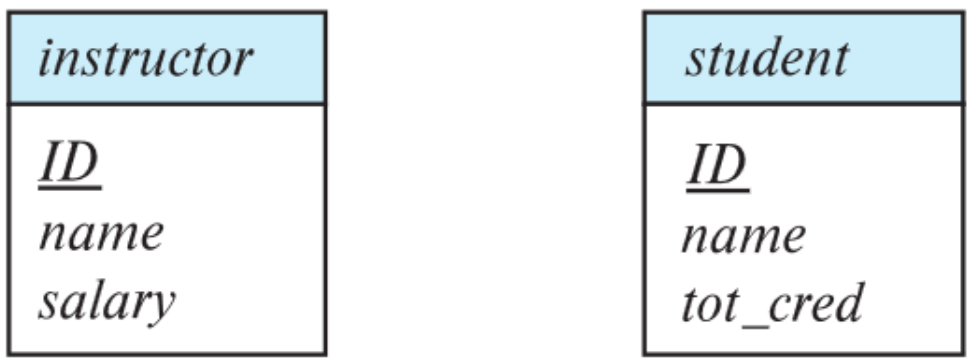
instructor

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

Representing Entity sets in ER Diagram

- Entity sets can be represented graphically as follows:
  - Rectangles represent entity sets.
  - Attributes listed inside entity rectangle
  - Underline indicates primary key attributes



Difference Between Entity, Entity Set and Entity Type

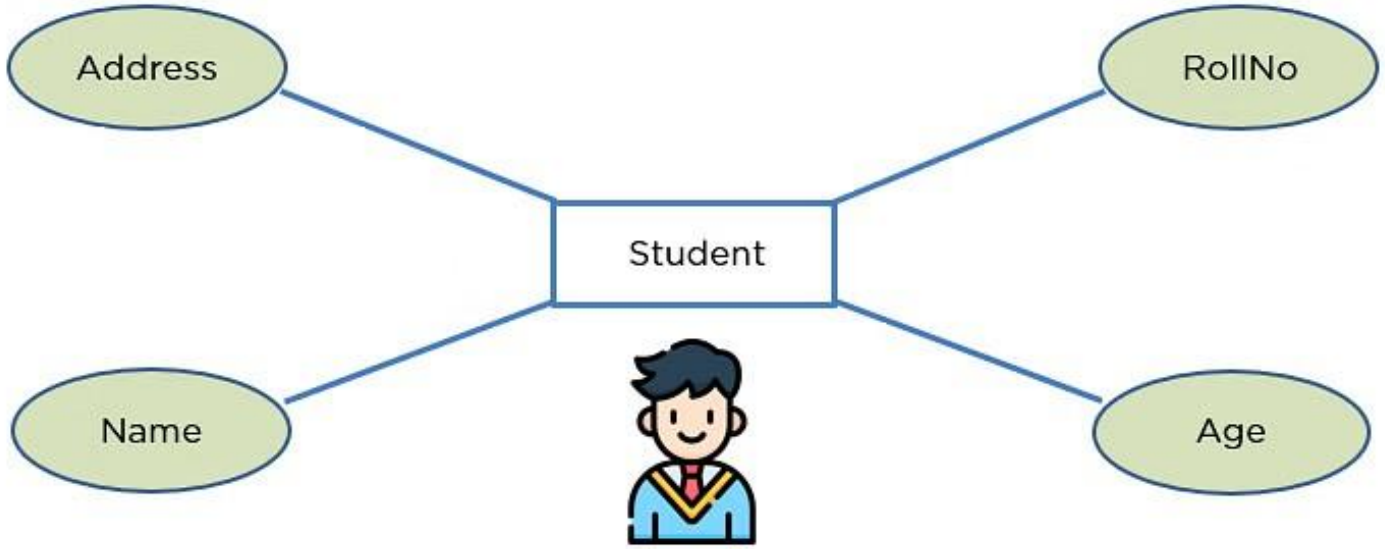
Entity	Entity Type	Entity Set
A thing in the real world with independent existence	A category of a particular entity	Set of all entities of a particular entity type.
Any particular row (a record) in a relation (table) is known as an entity.	The name of a relation (table) in RDBMS is an entity type	All rows of a relation (table) in RDBMS is entity set
Entities can be tangible or intangible.	Defines attributes shared by entities of that type.	Represents a snapshot of all entities at a given time.
It is identified uniquely through a key attribute.	It represents the structure of the table without data.	It can grow or shrink as entities are added or removed.
Example: A student with ID 1.	**Example:** “Student” table schema.	**Example:** All records in the “Student” table.

Attribute

Each entity is described by a set of attributes (e.g., Employee = (Name, Address, Birthdate (Age), Salary).

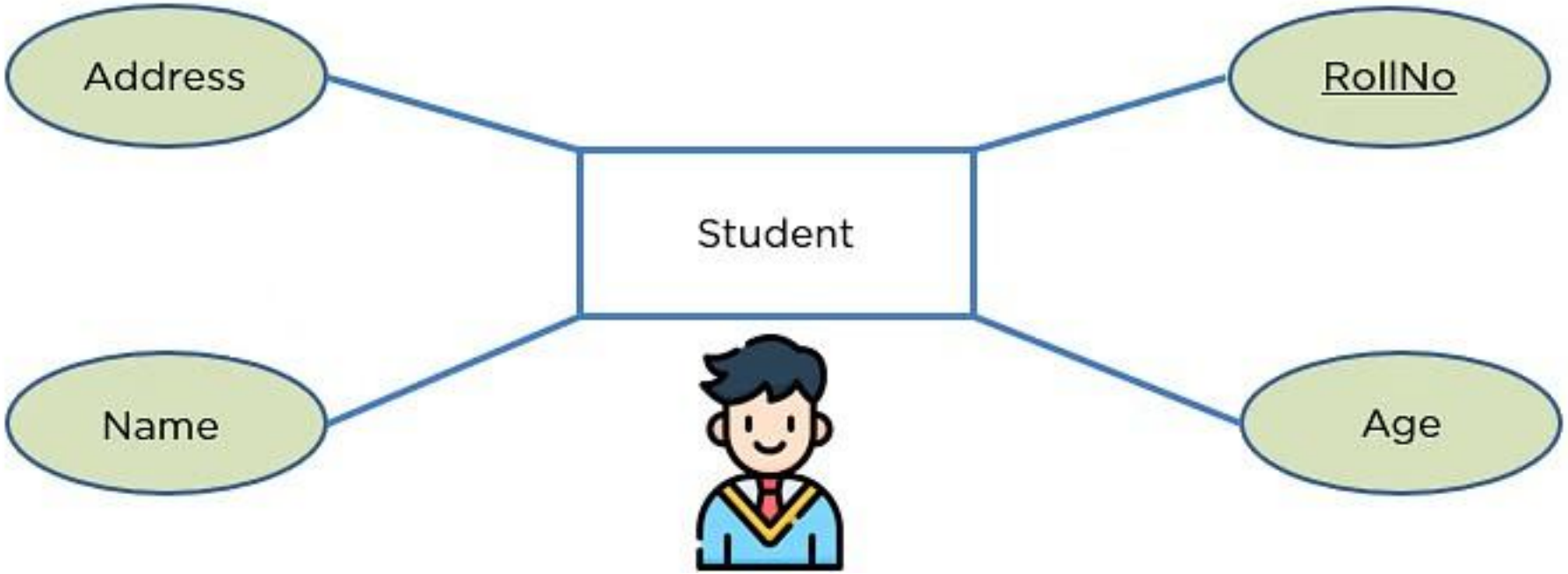
Each attribute has a name, and is associated with an entity and a domain of legal values.

An attribute is shown with an oval shape in an ER diagram.



Key Attribute

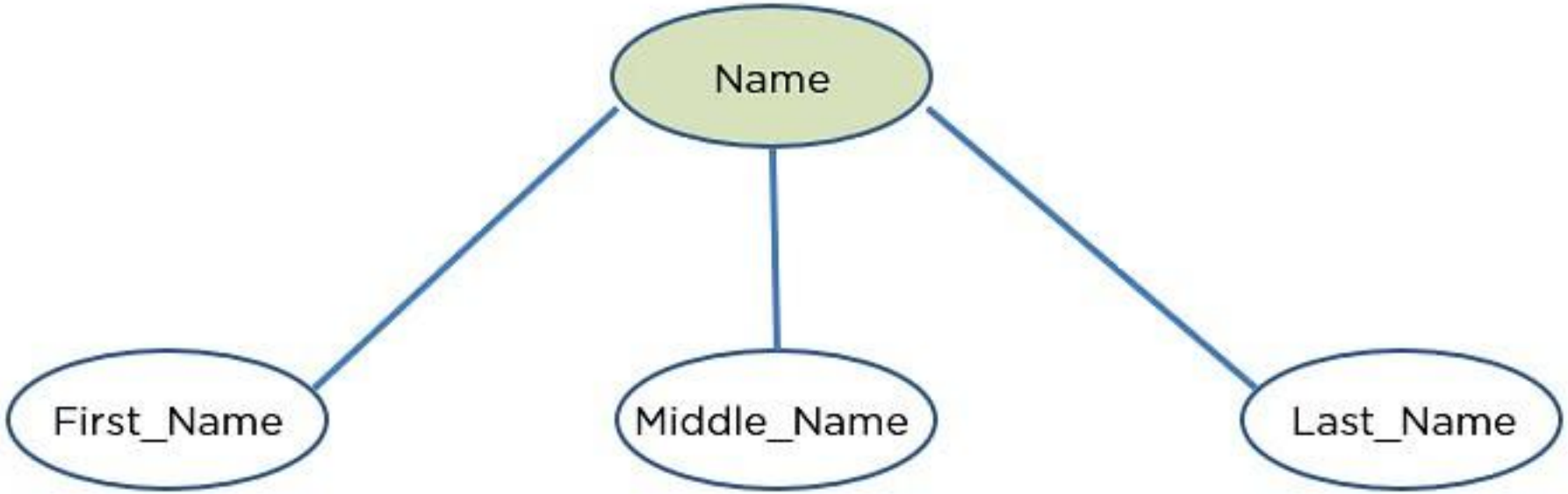
Key attribute uniquely identifies an entity from an entity set.
It underlines the text of a key attribute.
For example: For a student entity, the roll number can uniquely identify a student from a set of students.



Composite Attribute

An attribute that is composed of several other attributes is known as a composite attribute.

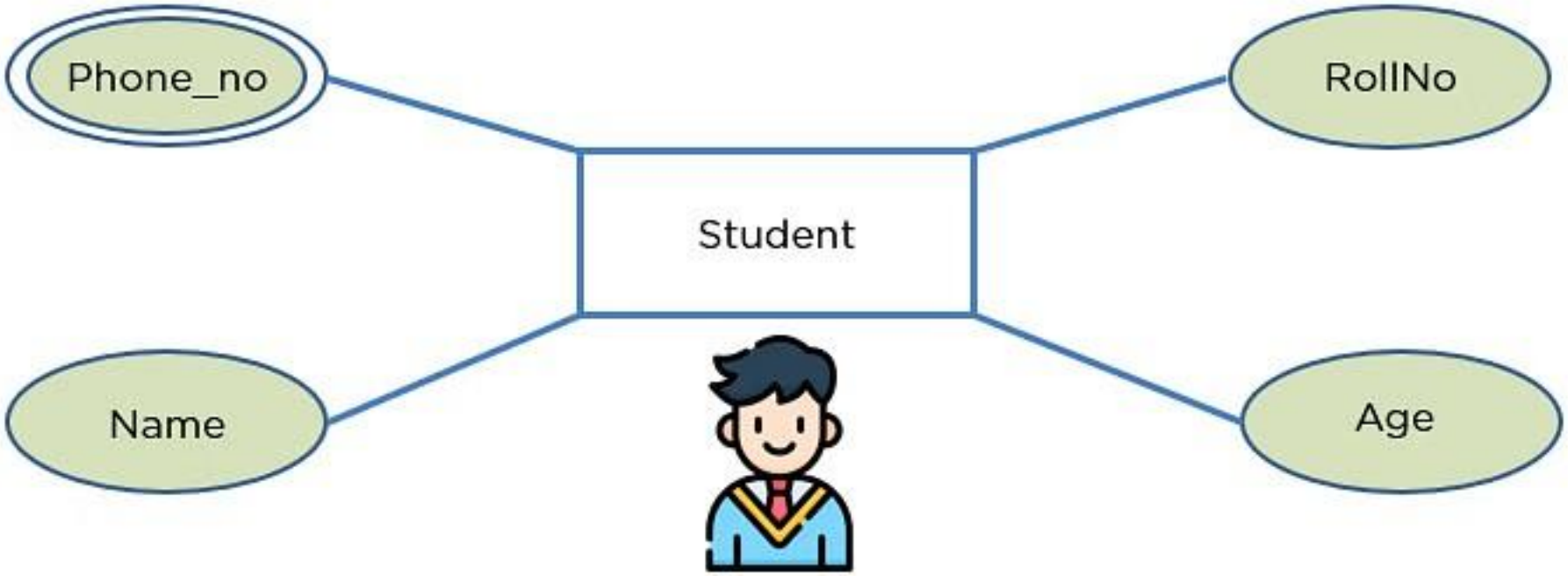
An oval showcases the composite attribute, and the composite attribute oval is further connected with other ovals.



Multivalued Attribute

Some attributes can possess over one value, those attributes are called multivalued attributes.

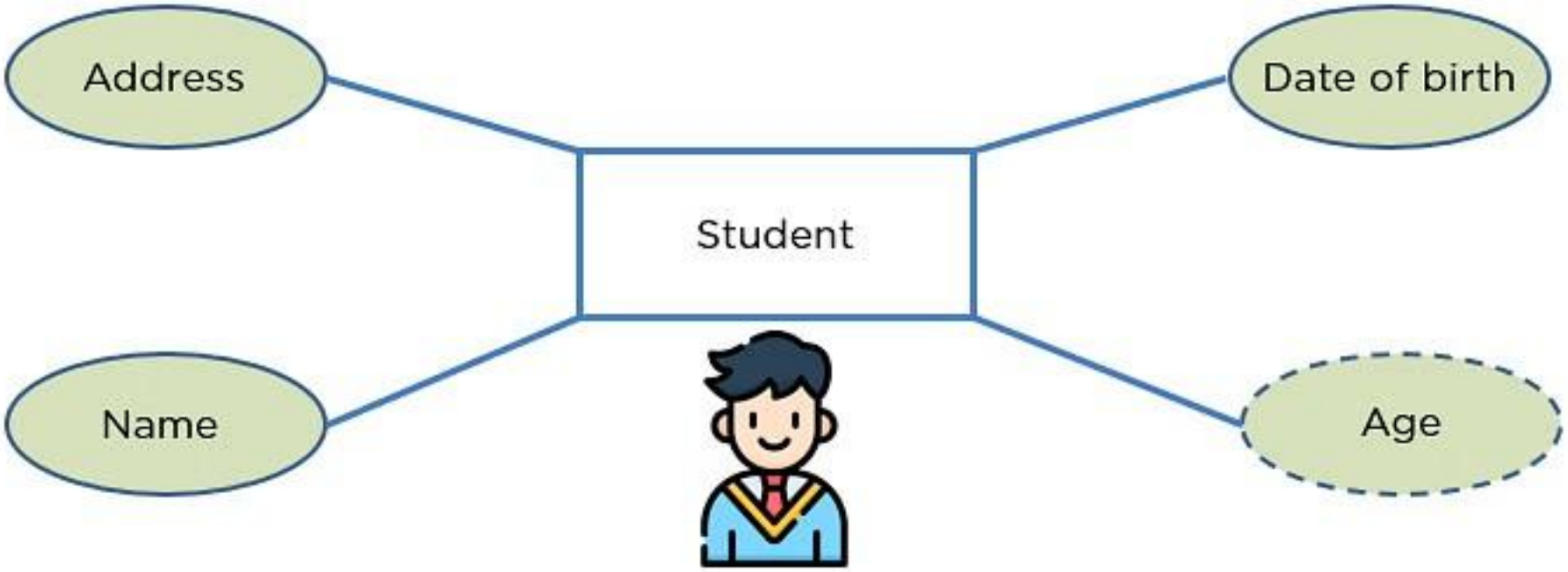
The double oval shape is used to represent a multivalued attribute.



Derived Attribute

An attribute that can be derived from other attributes of the entity is known as a derived attribute.

In the ER diagram, the dashed oval represents the derived attribute.



Candidate key

A *candidate key* is a simple or composite key that is unique and minimal. It is unique because no two rows in a table may have the same value at any time. It is minimal because every column is necessary in order to attain uniqueness.

From our UNIVERSITY database example, if the entity is **Faculty**(EID, First Name, Last Name, *SIN*, Address, Phone, BirthDate, Salary, DepartmentID), possible candidate keys are:

- EID, SIN
- First Name and Last Name – assuming there is no one else in the company with the same name
- Last Name and DepartmentID – assuming two people with the same last name don't work in the same department

Composite key

A *composite key* is composed of two or more attributes, but it must be minimal.

Using the example from the candidate key section, possible composite keys are:

- First Name and Last Name – assuming there is no one else in the company with the same name
- Last Name and Department ID – assuming two people with the same last name don’t work in the same department

Primary key

The primary key is a candidate key that is selected by the database designer to be used as an identifying mechanism for the whole entity set. It must uniquely identify tuples in a table and not be null. The primary key is indicated in the ER model by underlining the attribute.

- A candidate key is selected by the designer to uniquely identify tuples in a table. It must not be null.
- A key is chosen by the database designer to be used as an identifying mechanism for the whole entity set. This is referred to as the primary key. This key is indicated by underlining the attribute in the ER model.

In the following example, EID is the primary key:

Faculty(EID, First Name, Last Name, SIN, Address, Phone, BirthDate, Salary, DepartmentID)

Secondary key

A *secondary key* is an attribute used strictly for retrieval purposes (can be composite), for example: Phone and Last Name.

Alternate key

Alternate keys are all candidate keys not chosen as the primary key.

Foreign key

A *foreign key (FK)* is an attribute in a table that references the primary key in another table OR it can be null. Both foreign and primary keys must be of the same data type.

In the UNIVERSITY database example below, DepartmentID is the foreign key:

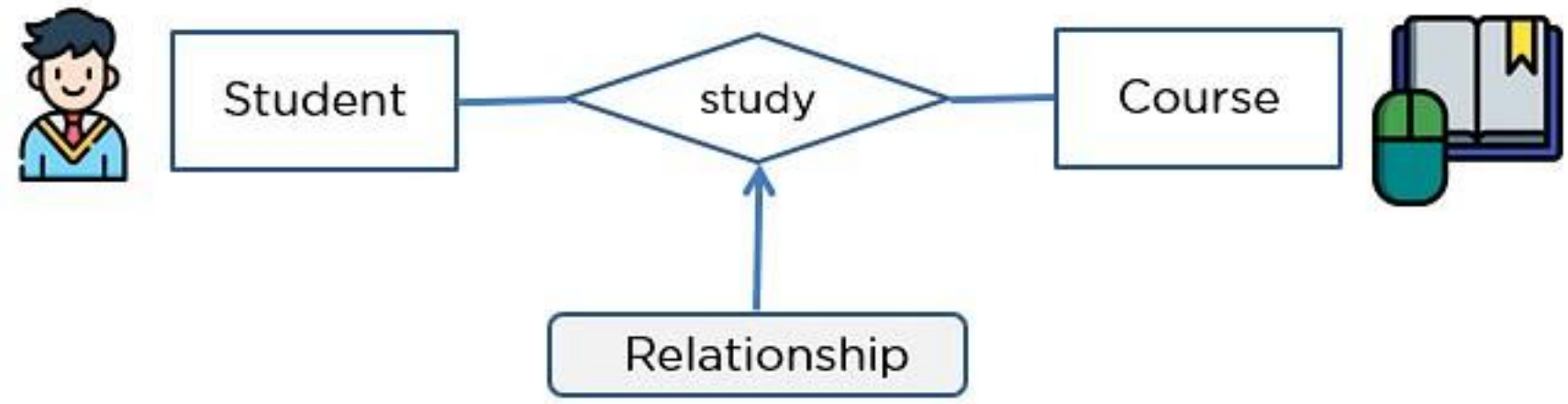
Faculty(EID, First Name, Last Name, SIN, Address, Phone, BirthDate, Salary, DepartmentID)

Relationship

The diamond shape showcases a relationship in the ER diagram.

It depicts the relationship between two entities.

In the example below, both the student and the course are entities, and study is the relationship between them.



Relationship strength is based on how the primary key of a related entity is defined. A weak, or non-identifying, relationship exists if the primary key of the related entity does not contain a primary key component of the parent entity.

A **weak relationship** exists when one entity is **dependent on another entity** (i.e., a weak entity).

- The weak entity **cannot be uniquely identified without** the strong entity.
- The weak entity has a **partial key** and uses a **foreign key from the strong entity** for identification.

Example: Relationship between Course and CourseSection (One-to-Many)

- **Entities:**
  - Course (Course_ID, Course_Name, Credits, Department) *(Strong Entity)*
  - CourseSection (Section_ID, Course_ID, Faculty_ID, Room_Number, Schedule) *(Weak Entity)*
- **Relationship:**
  - A **course has multiple sections**, but **a section cannot exist without a course**.
  - CourseSection is a **weak entity** because Section_ID alone is not enough to identify it uniquely; it depends on Course_ID.

Strong Relationship

- A **strong relationship** exists when two **independent (strong) entities** are related.
- The related entities have **primary keys** that uniquely identify them.
- The relationship **does not require** a weak entity to exist.

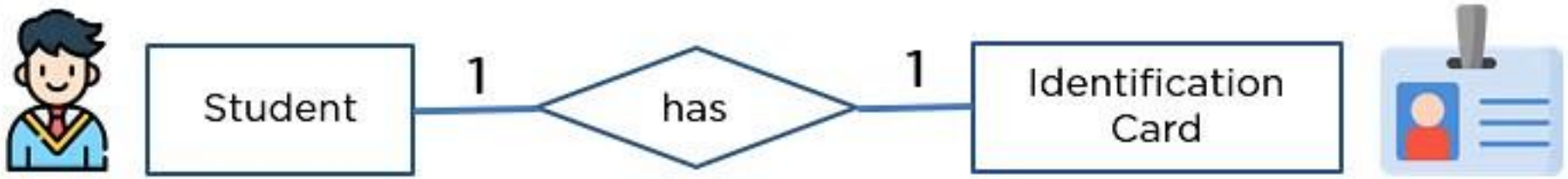
Example: Relationship between Student and Course (Many-to-Many)

- **Entities:**
  - Student (Student_ID, Name, Email, Major)
  - Course (Course_ID, Course_Name, Credits, Department)
- **Relationship:**
  - A **student enrolls in multiple courses**, and a **course has multiple students enrolled**.
  - This **many-to-many relationship** is **strong** because both Student and Course are independent entities.

One-to-One Relationship

When a single element of an entity is associated with a single element of another entity, it is called a one-to-one relationship.

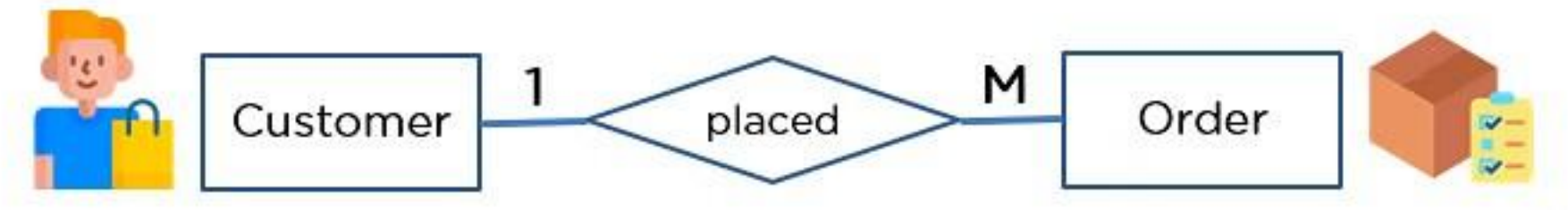
For example, a student has only one identification card and an identification card is given to one person.



One-to-Many Relationship

When a single element of an entity is associated with more than one element of another entity, it is called a one-to-many relationship

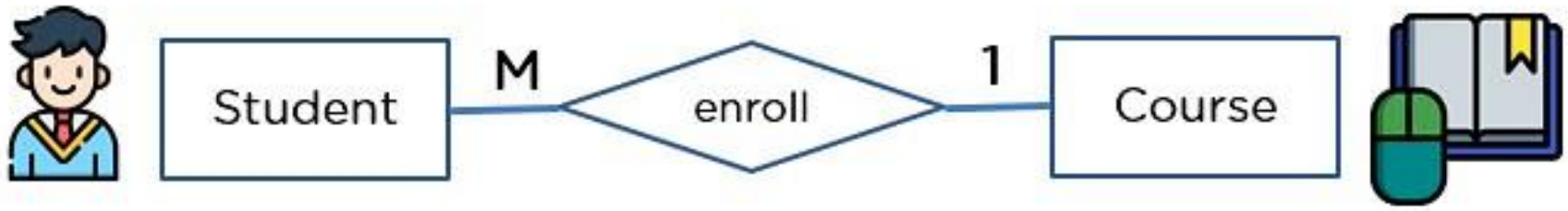
For example, a customer can place many orders, but an order cannot be placed by many customers.



Many-to-One Relationship

When more than one element of an entity is related to a single element of another entity, then it is called a many-to-one relationship.

For example, students have to opt for a single course, but a course can have many students.



Many-to-Many Relationship

When more than one element of an entity is associated with more than one element of another entity, this is called a many-to-many relationship.

For example, you can assign an employee to many projects and a project can have many employees.

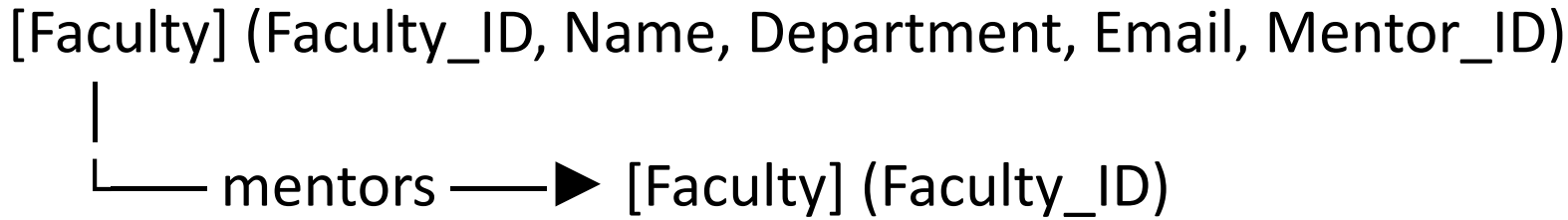


Unary relationship (recursive)

A *unary relationship,* also called *recursive,* is one in which a relationship exists between occurrences of the same entity set. In this relationship, the primary and foreign keys are the same, but they represent two entities with different roles.

Example: Faculty Mentorship (Faculty → Faculty)

- A faculty member **mentors** another faculty member (e.g., senior professors mentoring junior faculty).
- This creates a **one-to-many unary relationship**.



Ternary Relationships

A *ternary relationship* is a relationship type that involves many to many relationships between three tables.

It often requires an **associative (junction) table** to store the relationship.

Example: Faculty Teaching Courses in a Semester (Faculty → Course → Semester)

- A **faculty member teaches a course in a specific semester**.
- This creates a **ternary (three-way) relationship**.

[Faculty] —— teaches —— [Course] —— offered in —— [Semester]

What is a Relationship Set?

A relationship set, also known as an association or a link, defines how two or more relations are related to each other within a database. It is the means by which data from different tables can be connected and queried together.

Characteristics of a Relationship Set:

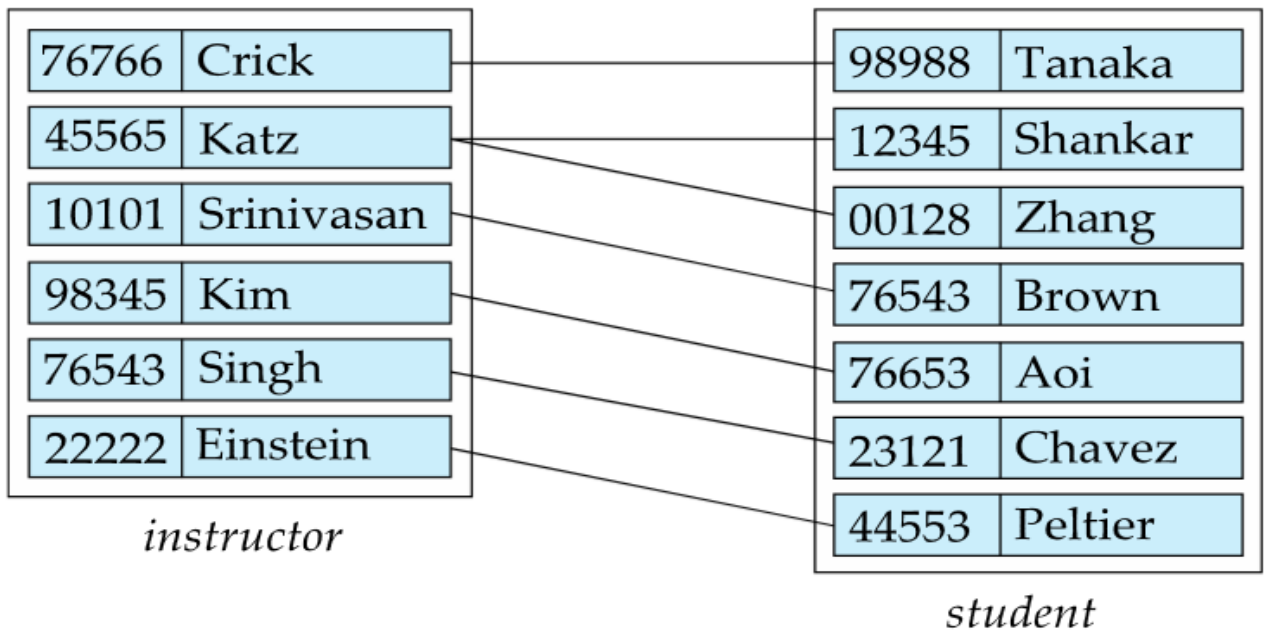
1.Arity: The arity of a relationship set specifies the number of participating relations. It can be binary (involving two relations), ternary (involving three relations), and so on.

2.Degree: The degree of a relationship set refers to the number of attributes associated with the relationship set.

3.Cardinality: Cardinality defines the number of instances or records that can be associated with each entity on both sides of the relationship. It is classified into one-to-one, one-to-many, or many-to-many cardinality.

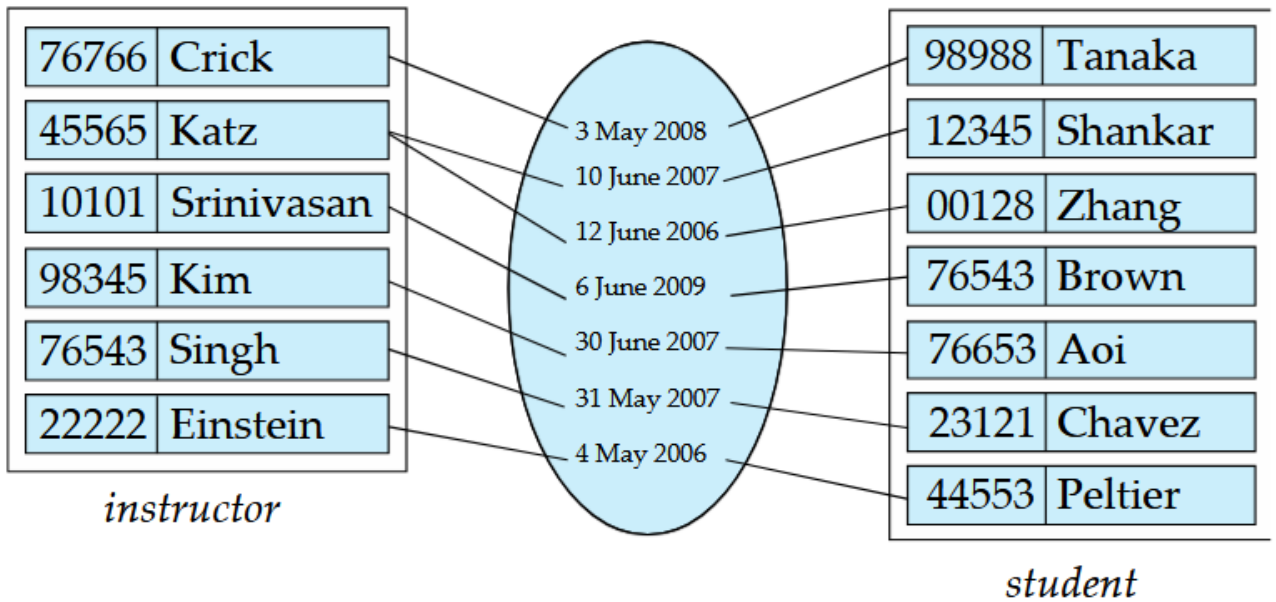
Relationship Sets (Cont.)

- Example: we define the relationship set *advisor* to denote the associations between students and the instructors who act as their advisors.
- Pictorially, we draw a line between related entities.



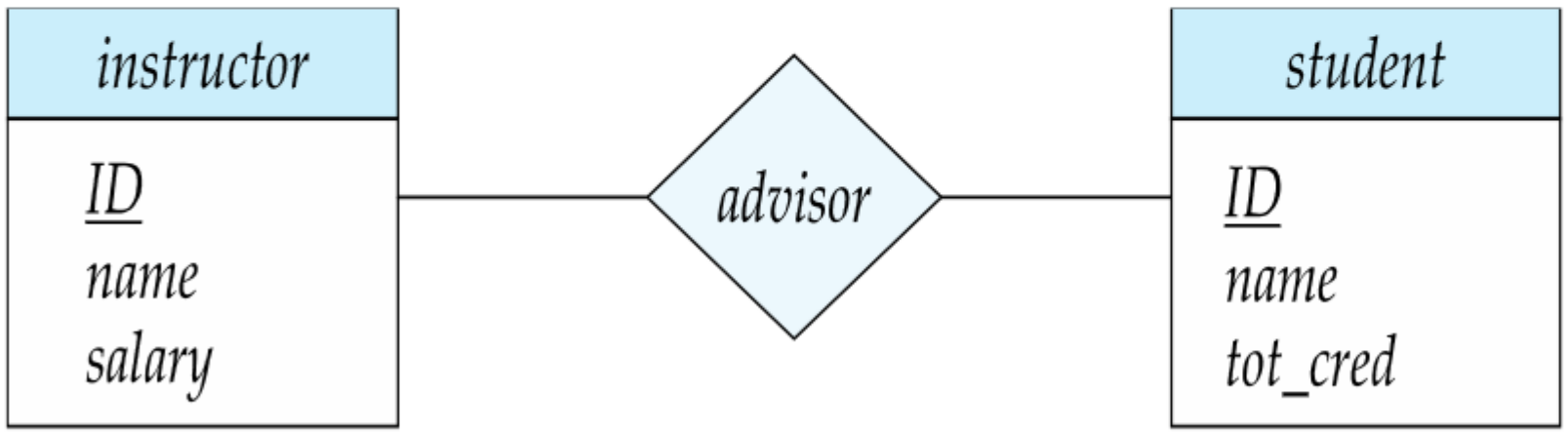
Relationship Sets (Cont.)

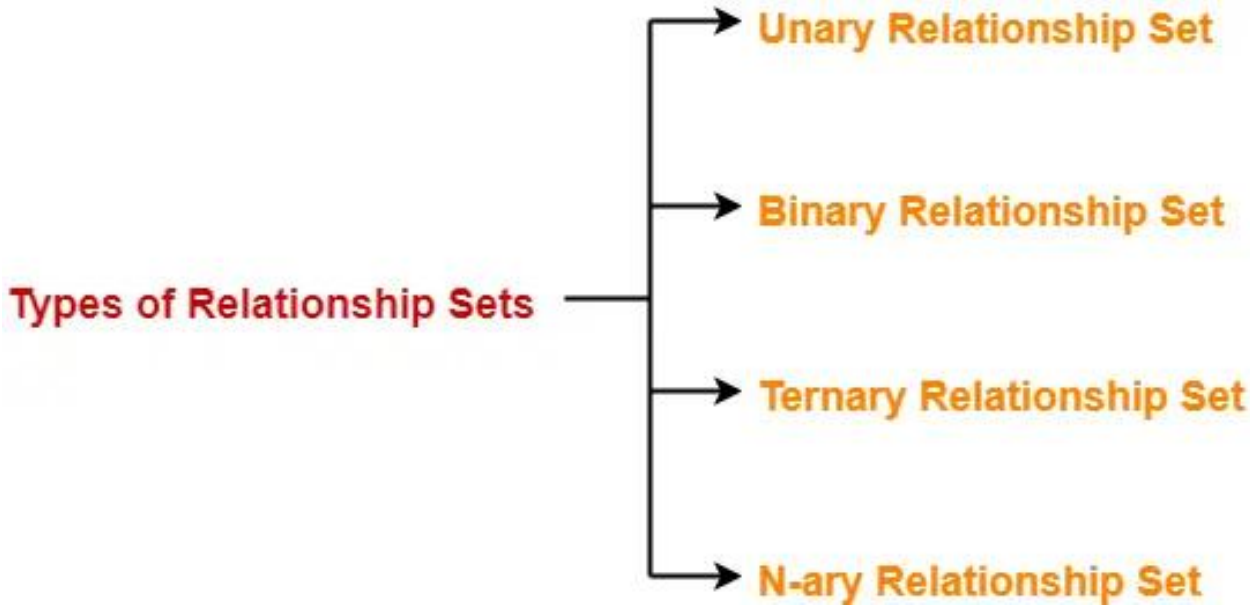
- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor

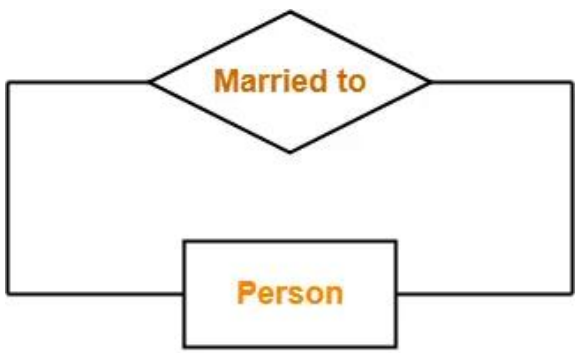


Representing Relationship Sets via ER Diagrams

- Diamonds represent relationship sets.



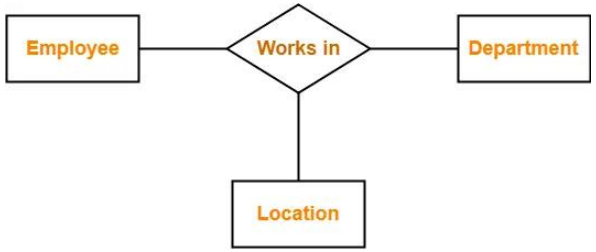




Unary Relationship Set

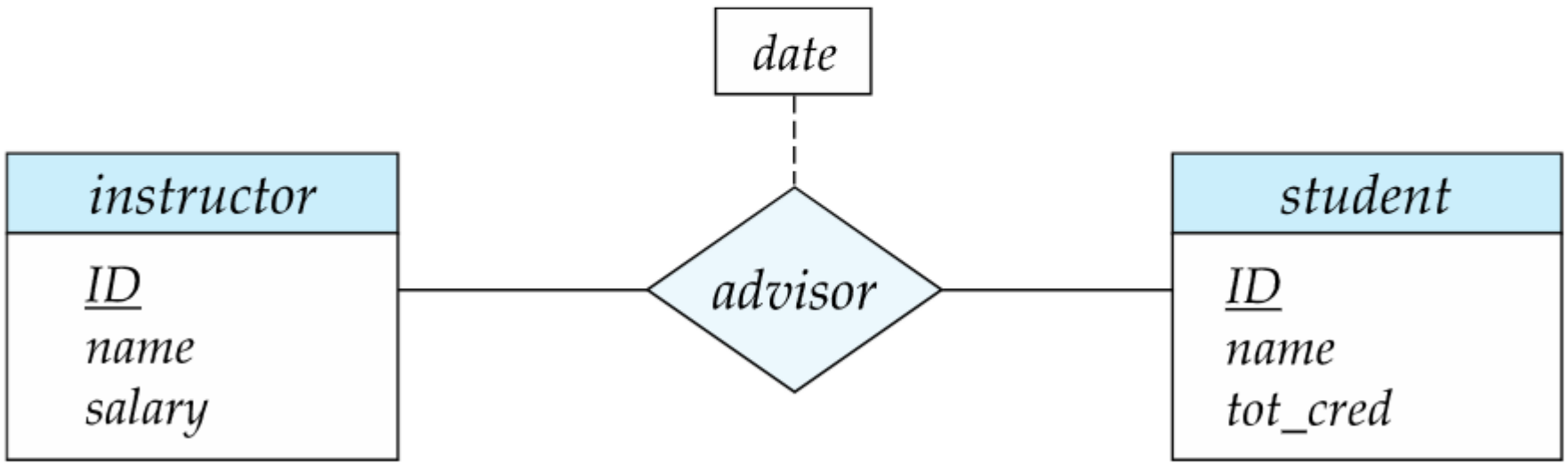


Binary Relationship Set



Ternary Relationship Set

Relationship Sets with Attributes



Cardinality

Cardinality means how the entities are arranged to each other or what is the relationship structure between entities in a relationship set. In a Database Management System, Cardinality represents a number that denotes how many times an entity is participating with another entity in a relationship set.

Cardinality Ratio

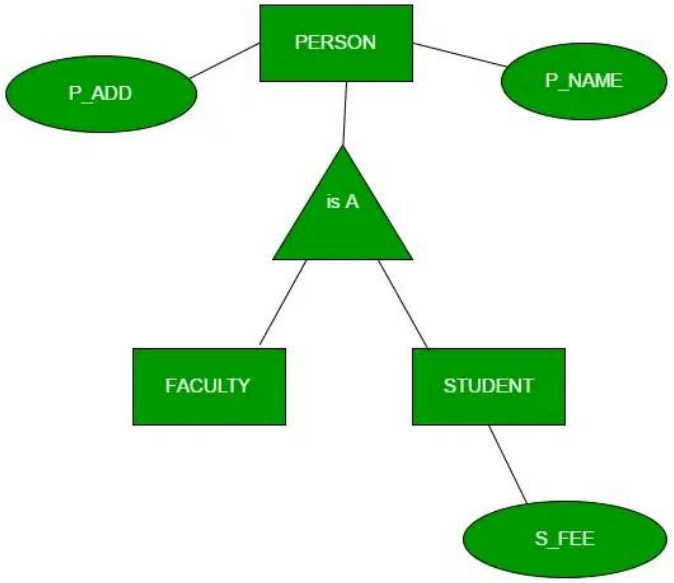
Cardinality ratio is also called **Cardinality Mapping**, which represents the mapping of one entity set to another entity set in a relationship set. We generally take the example of a binary relationship set where two entities are mapped to each other.

Additional Features of ER Model

Generalization

Generalization is the process of extracting common properties from a set of entities and creating a generalized entity from it. It is a bottom-up approach in which two or more entities can be generalized to a higher-level entity if they have some attributes in common. For Example, STUDENT and FACULTY can be generalized to a higher-level entity called PERSON as shown in Figure 1. In this case, common attributes like P_NAME, and P_ADD become part of a higher entity (PERSON), and specialized attributes like S_FEE become part of a specialized entity (STUDENT).

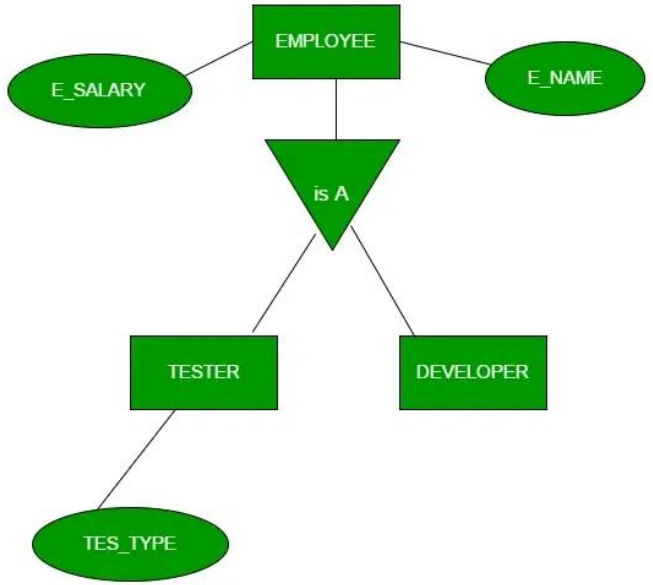
Generalization is also called as ‘ Bottom-up approach”.



Specialization

In specialization, an entity is divided into sub-entities based on its characteristics. It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities. For Example, an EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER, etc. as shown in Figure 2. In this case, common attributes like E_NAME, E_SAL, etc. become part of a higher entity (EMPLOYEE), and specialized attributes like TES_TYPE become part of a specialized entity (TESTER).

Specialization is also called as ” Top-Down approch”.

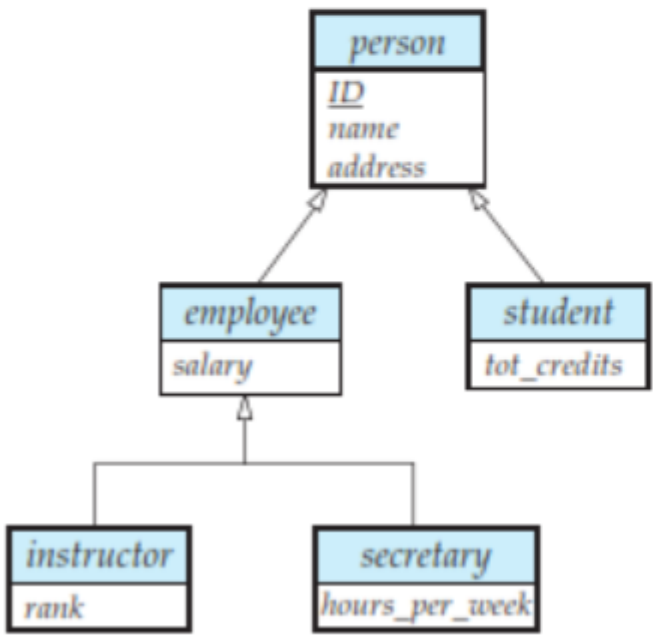


Specialization

- Types of specialization :

    1. **overlapping specialization :** An entity may belong to multiple specialized entity sets.

    2. **disjoint specialization :** An entity may belong to at most one specialized entity sets.



Specialization and generalization.

Representing Specialization via Schemas

- Method 1:
  - Form a schema for the higher-level entity
  - Form a schema for each lower-level entity set, include primary key of higher-level entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

  - Drawback: getting information about, an *employee* requires accessing two relations, the one corresponding to the low-level schema and the one corresponding to the high-level schema

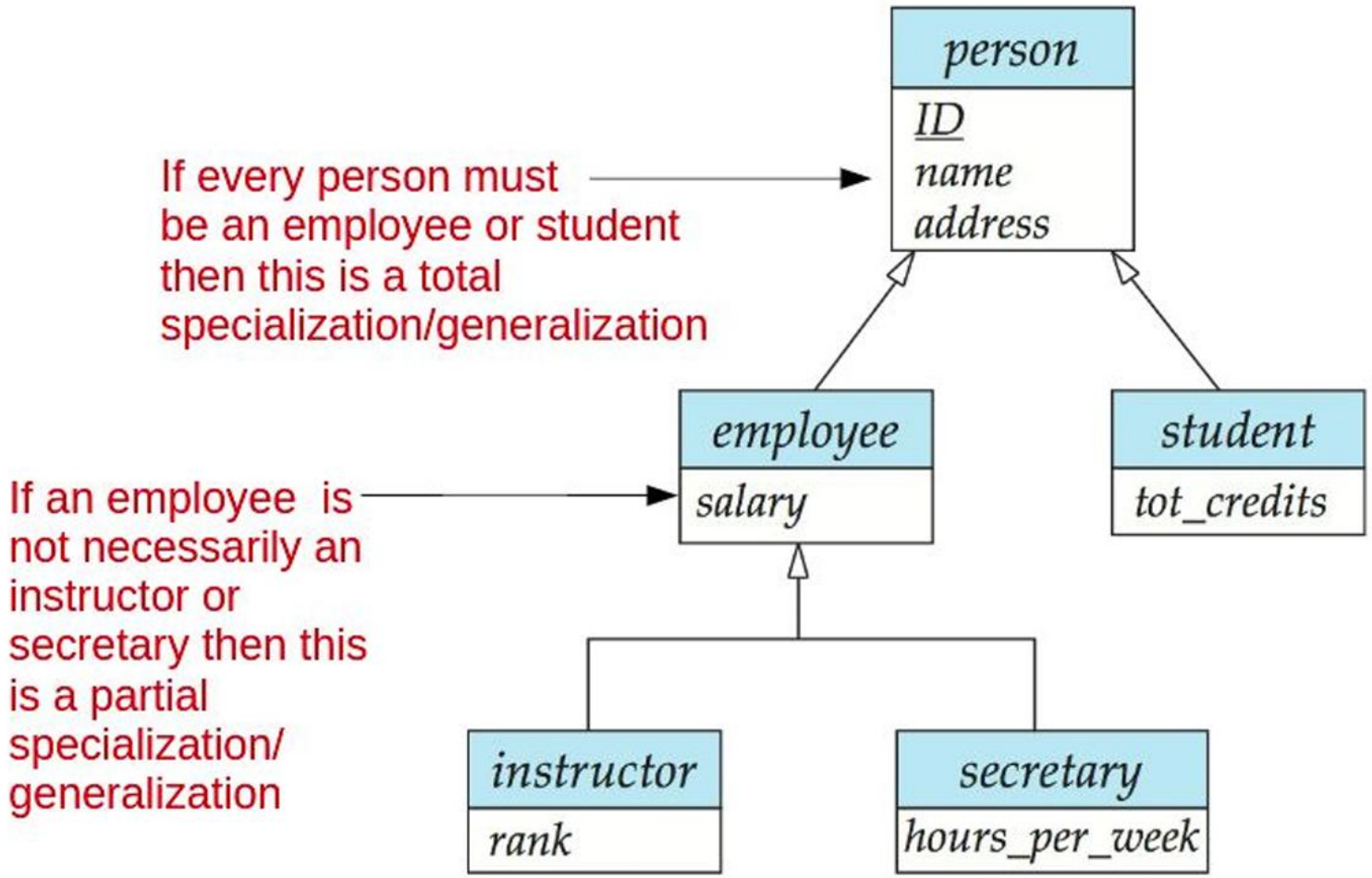
Representing Specialization as Schemas (Cont.)

- Method 2:
  - Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

  - Drawback: *name, street* and *city* may be stored redundantly for people who are both students and employees

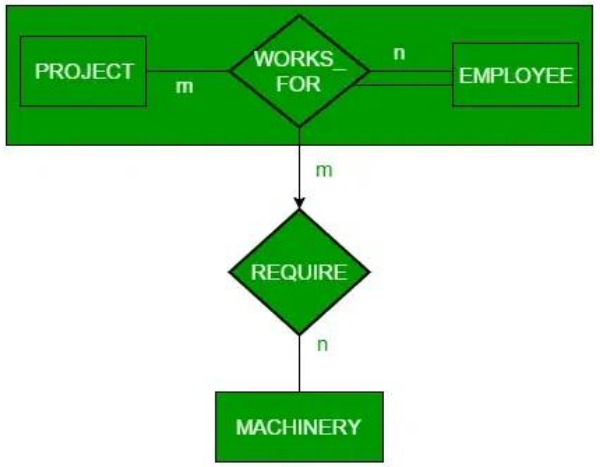
Specialization/Generalization Design Constraints



Aggregation

An ER diagram is not capable of representing the relationship between an entity and a relationship which may be required in some scenarios. In those cases, a relationship with its corresponding entities is aggregated into a higher-level entity. Aggregation is an abstraction through which we can represent relationships as higher-level entity sets.

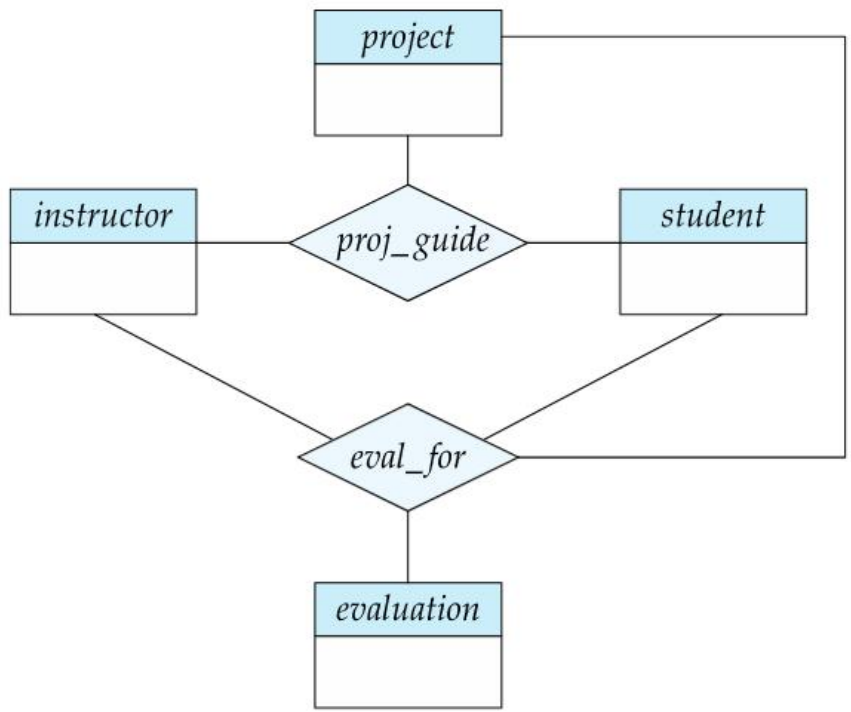
For Example, an Employee working on a project may require some machinery. So, REQUIRE relationship is needed between the relationship WORKS_FOR and entity MACHINERY. Using aggregation, WORKS_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into a single entity and relationship REQUIRE is created between the aggregated entity and MACHINERY.



Aggregation

Aggregation

- Consider the ternary relationship *proj_guide*, which we saw earlier
- Suppose we want to record evaluations of a student by a guide on a project



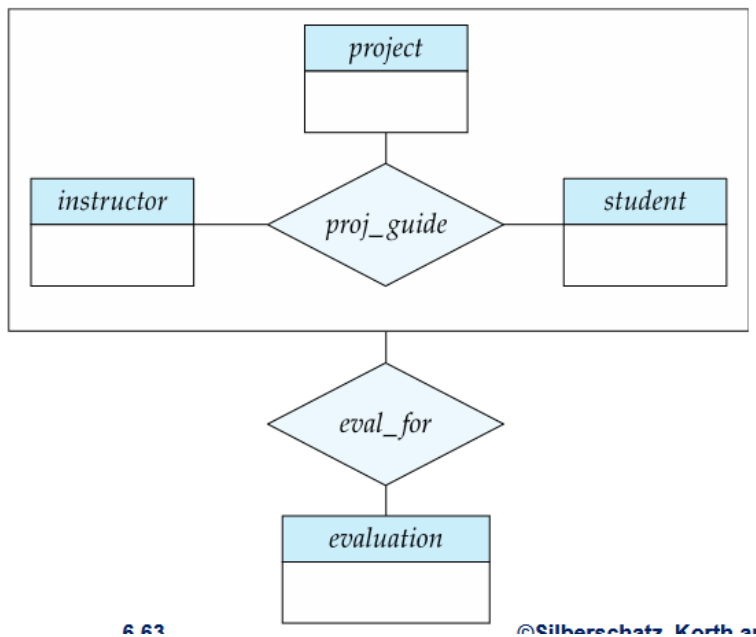
Aggregation (Cont.)

- Relationship sets *eval_for* and *proj_guide* represent overlapping information
  - Every *eval_for* relationship corresponds to a *proj_guide* relationship
  - However, some *proj_guide* relationships may not correspond to any *eval_for* relationships
    - So we can't discard the *proj_guide* relationship

- Eliminate this redundancy via *aggregation*
  - Treat relationship as an abstract entity
  - Allows relationships between relationships
  - Abstraction of relationship into new entity

Aggregation (Cont.)

- Eliminate this redundancy via *aggregation* without introducing redundancy, the following diagram represents:
  - A student is guided by a particular instructor on a particular project
  - A student, instructor, project combination may have an associated evaluation



Reduction to Relational Schemas

- To represent aggregation, create a schema containing
  - Primary key of the aggregated relationship,
  - The primary key of the associated entity set
  - Any descriptive attributes

- In our example:
  - The schema *eval_for* is:

    eval_for (*s_ID, project_id, i_ID, evaluation_id*)

Conceptual Design using ER Model

Steps in Designing an Entity-Relationship Schema

[Step 1] Identify entity types (entity type vs. attribute)

[Step 2] Identify relationship types

[Step 3] Identify and associate attributes with entity and relationship types

[Step 4] Determine attribute domains

[Step 5] Determine primary key attributes for entity types

[Step 6] Associate (refined) cardinality ratio(s) with relationship types

[Step 7] Design generalization/specialization hierarchies including constraints (includes natural language statements as well)

Translation of ER Schema into Tables

- An ER schema can be represented by a collection of tables which represent contents of the database (instance).
- Primary keys allow entity types and relationship types to be expressed uniformly as tables.
- For each entity and relationship type, a unique table can be derived which is assigned the name of the corresponding entity or relationship type.
- Each table has a number of columns that correspond to the attributes and which have unique names. An attribute of a table has the same domain as the attribute in the ER schema.
- Translating an ER schema into a collection of tables is the basis for deriving a relational database schema from an ER diagram.

Translating Entity Types into Tables

- Given an entity type E1 with (atomic) attributes A1, . . . , An and associated domains D1, . . . , Dn.
- Translation of the entity type CUSTOMERS into table CUSTOMERS

FName	LName	CAddress	Account
Michael	Smith	San Francisco	10,000
George	Jones	New York	2,000
Kent	Clark	Los Angeles	-4,600
. . .	. . .	. . .	. . .

In SQL:

```
create table Customers(
FName char(40),
LName char(40),
CAddress char(70),
Account real
);
```

- A row in such a table corresponds to an entity from the entity set.

Translating Relationship Types into Tables

• A many-many relationship type is represented as a table with columns for the primary key attributes of the participating entity types, and any descriptive attributes of the relationship type.

Example: Relationship type offers

Prodname	SName	Price
PC42	Hal-Mart	2,100
MacIV	Sears	2,500
. . .	. . .	. . .

Prodname and SName are the primary key attributes of the entity types SUPPLIERS and PRODUCTS.

• Translation of one-many and many-one (functional) and one-one relationship types into tables can be optimized ; no table for relationship type necessary!

Translating Subclasses/Superclasses into Tables

• Method 1: Form a table for the superclass and form a table for each subclass. Include the primary key attributes of the superclass in each such table.

Example:

Employees(SocialSN, Name, Address)
Hourly_Emps(SocialSN, Hours, Wages)
Contract_Emps(SocialSN, ContractNo)

• Method 2: Form a table for each subclass and include all attributes of the superclass.

Hourly_Emps(SocialSN, Name, Address, Hours, Wages)
Contract_Emps(SocialSN, Name, Address, Contractno)

Method 2 has no table for the superclass EMPLOYEES.

- Method 3: Use null values

  Employees(SocialSN, Name, Address, Hours, Wages, ContractNo)

Hourly employees will have a null value for ContractNo.

Contract employees will have null values for Hours and Wages.

When not to use a DBMS

- Main inhibitors (costs) of using a DBMS:
  - High initial investment and possible need for additional hardware
  - Overhead for providing generality, security, concurrency control, recovery, and integrity functions
- When a DBMS may be unnecessary:
  - If the database and applications are simple, well defined, and not expected to change
  - If access to data by multiple users is not required
- When a DBMS may be infeasible
  - In embedded systems where a general-purpose DBMS may not fit in available storage

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

When not to use a DBMS

- When no DBMS may suffice:
  - If there are stringent real-time requirements that may not be met because of DBMS overhead (e.g., telephone switching systems)
  - If the database system is not able to handle the complexity of data because of modeling limitations (e.g., in complex genome and protein databases)
  - If the database users need special operations not supported by the DBMS (e.g., GIS and location-based services).

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe